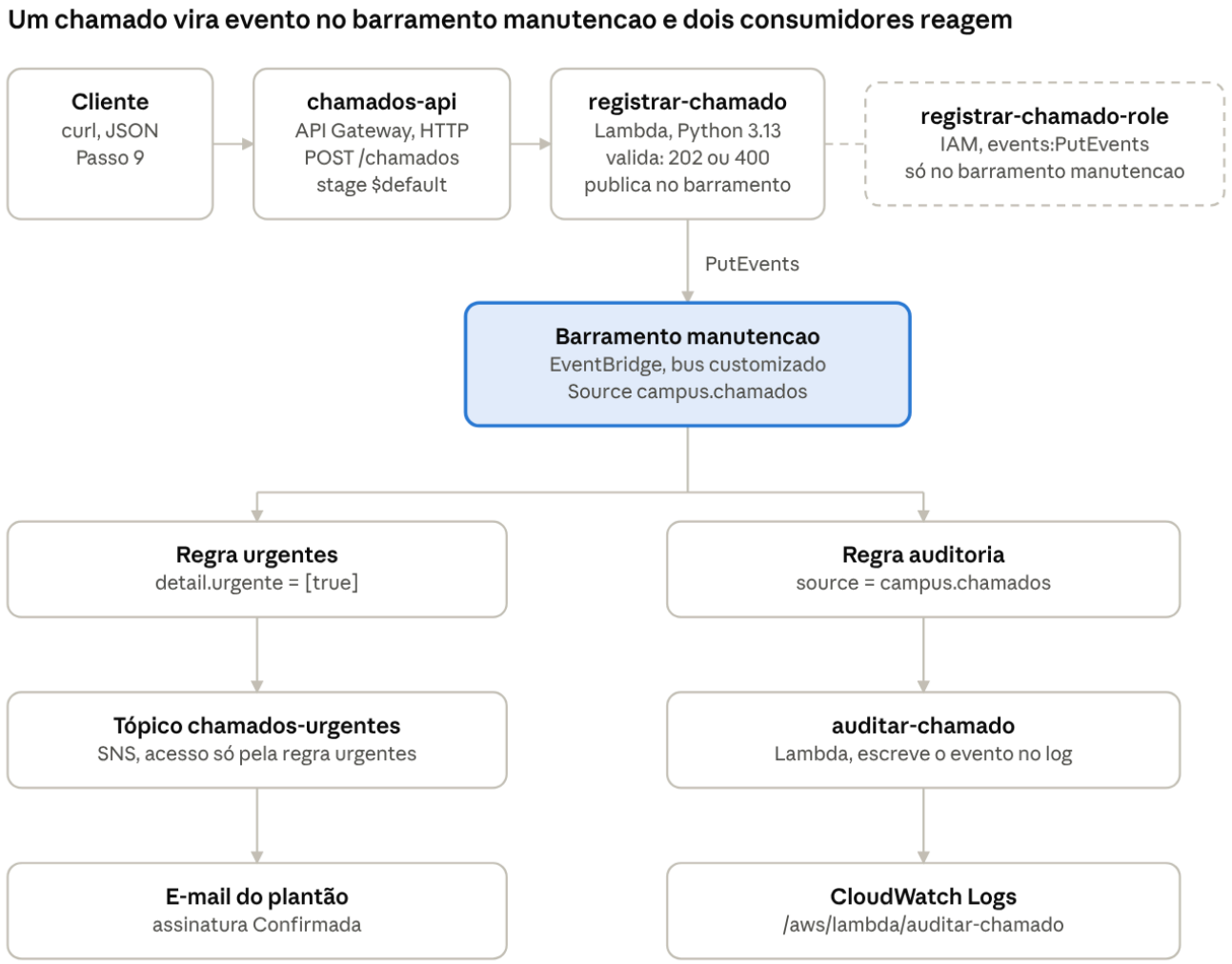
Entrega: Central de Chamados de Manutenção (Aula 11)

29 de outubro de 2026

Grupo: João Henrique Micucci · João Pedro Mamede · João Pedro Morais · Caio Guimarães · Matheus Lustosa · Rafael Serpa

Central de chamados orientada a eventos na AWS (região us-east-1), criada com Terraform: API Gateway, Lambda, EventBridge, SNS e CloudWatch Logs. Todas as capturas vêm do console da AWS, da caixa de entrada e, onde indicado, do terminal.

1. Arquitetura



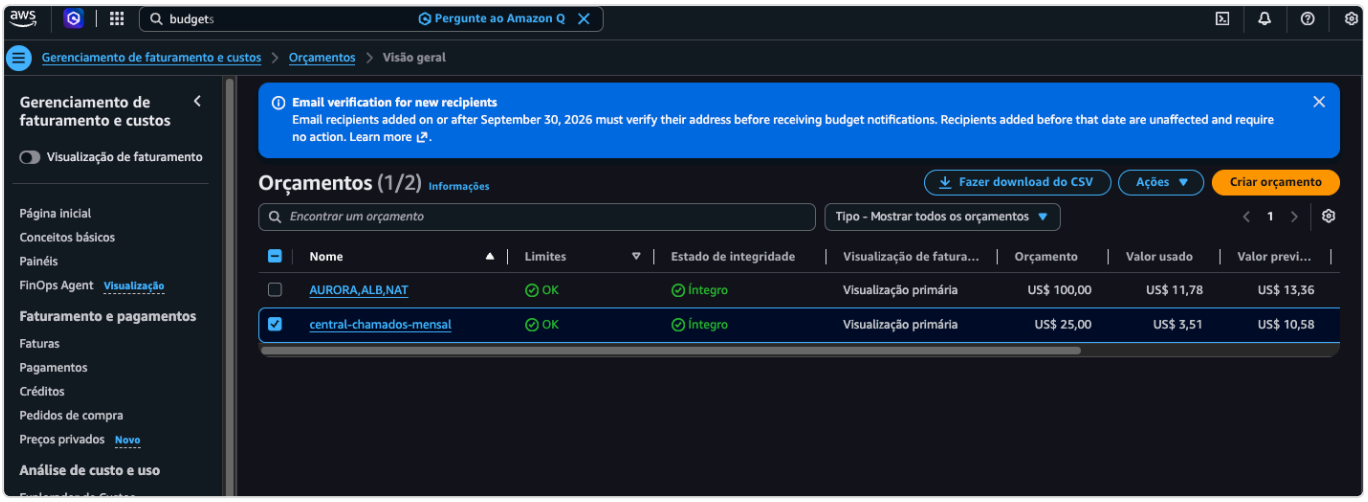
Arquitetura da central de chamados · recursos reais da conta, us-east-1

O cliente chama `POST /chamados`; a função `registrar-chamado` valida e publica o evento `ChamadoRegistrado` no barramento `manutencao`, e o barramento entrega por duas regras: `urgentes` (e-mail pelo SNS) e `auditoria` (log pela função `auditar-chamado`). A função de auditoria tem a própria role, `auditar-chamado-role`.

2. Recursos criados

Todos os recursos nasceram do Terraform, na região us-east-1 (N. Virginia), e aparecem no console como descrito abaixo.

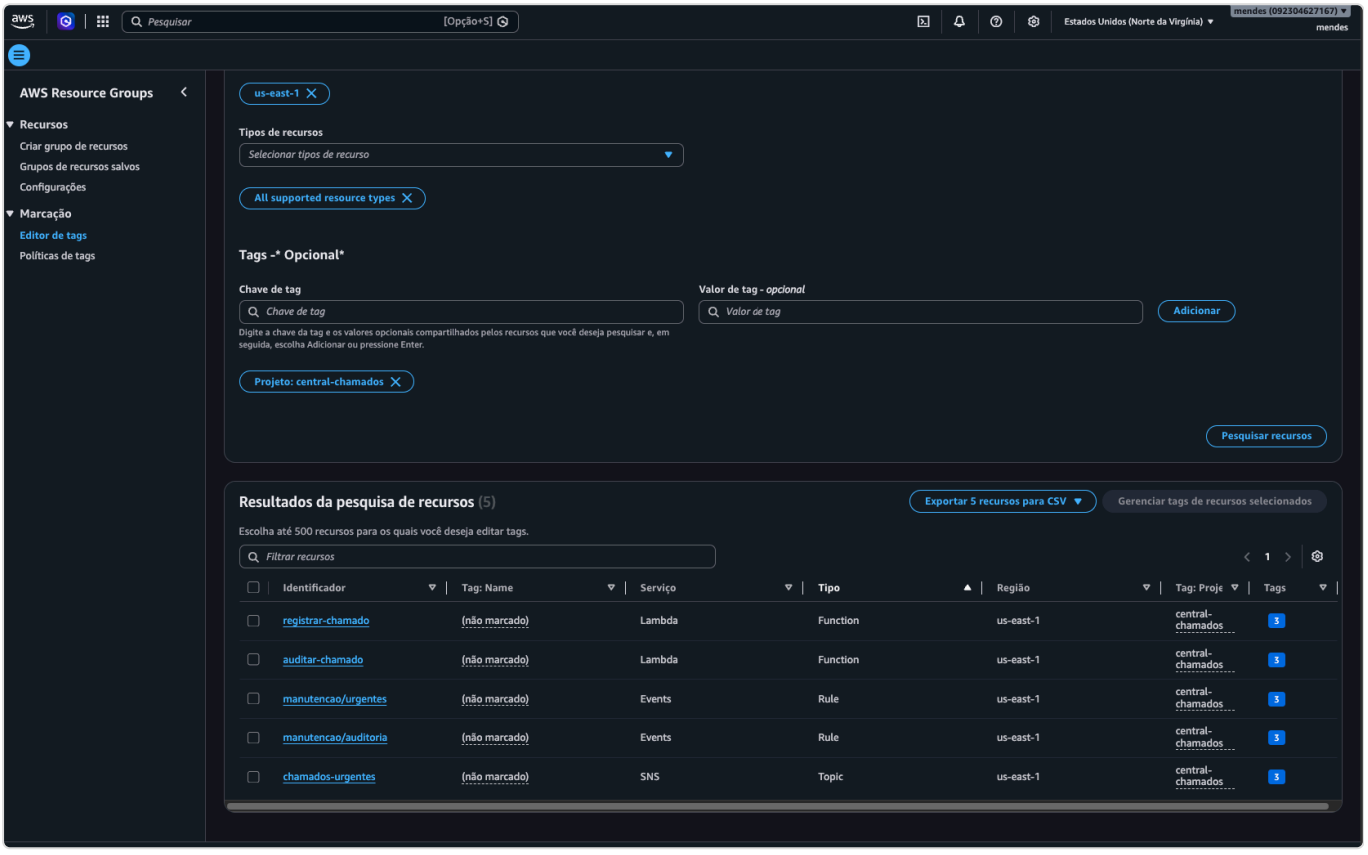
Orçamento criado antes de qualquer recurso



E01 · Orçamento central-chamados-mensal, criado antes dos demais recursos (Passo 0)

O alerta de custo `central-chamados-mensal` existe antes da infraestrutura, com limite mensal de US$ 25,00 e estado Íntegro. O valor usado inclui outros recursos da conta (o orçamento `AURORA,ALB,NAT` é de outra prática).

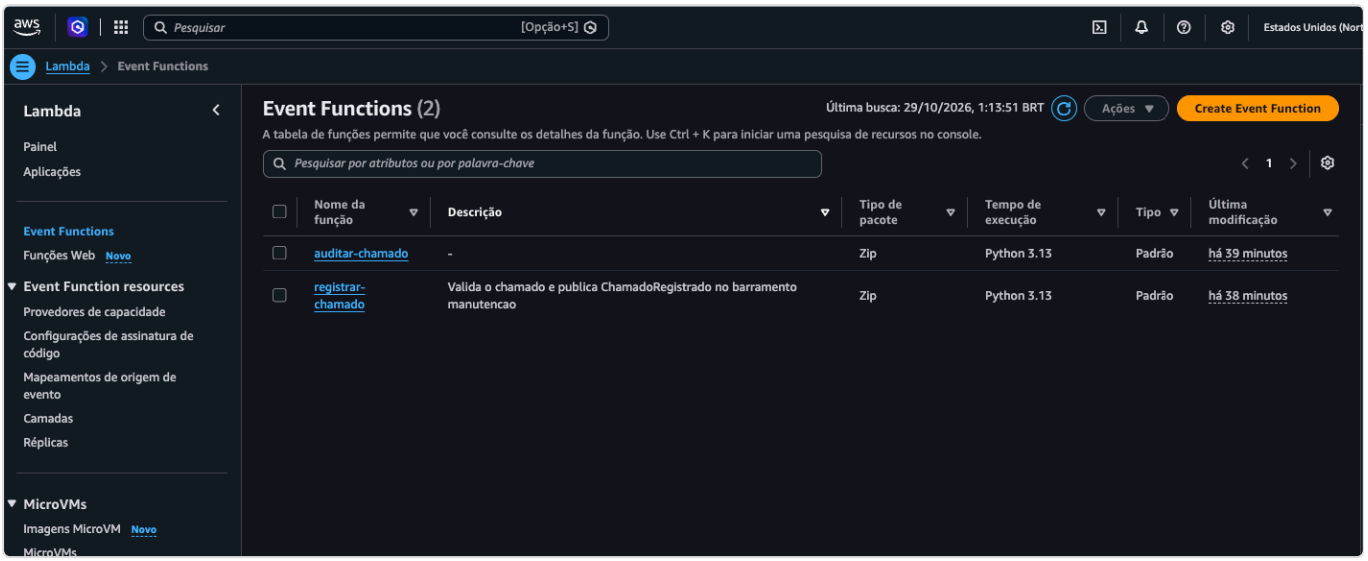
Inventário por tag



E02 · Resource Groups e Tag Editor: recursos com a tag Projeto = central-chamados

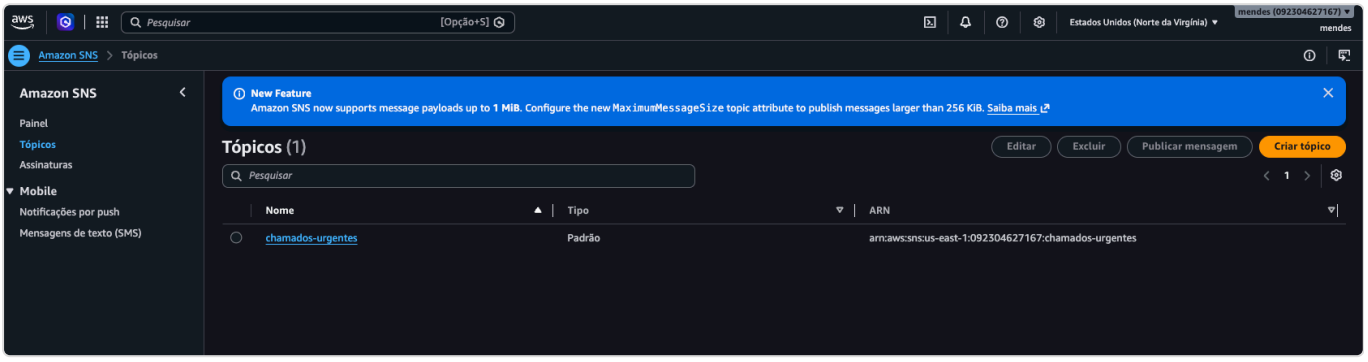
Cinco recursos levam a tag `Projeto: central-chamados`: as funções `registrar-chamado` e `auditar-chamado`, as regras `manutencao/urgentes` e `manutencao/auditoria` e o tópico SNS `chamados-urgentes`.

As duas funções Lambda

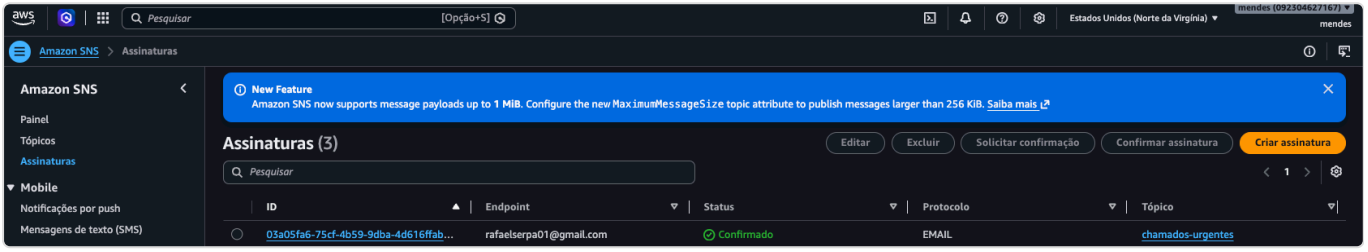


E03 · Funções registrar-chamado e auditar-chamado, runtime Python 3.13

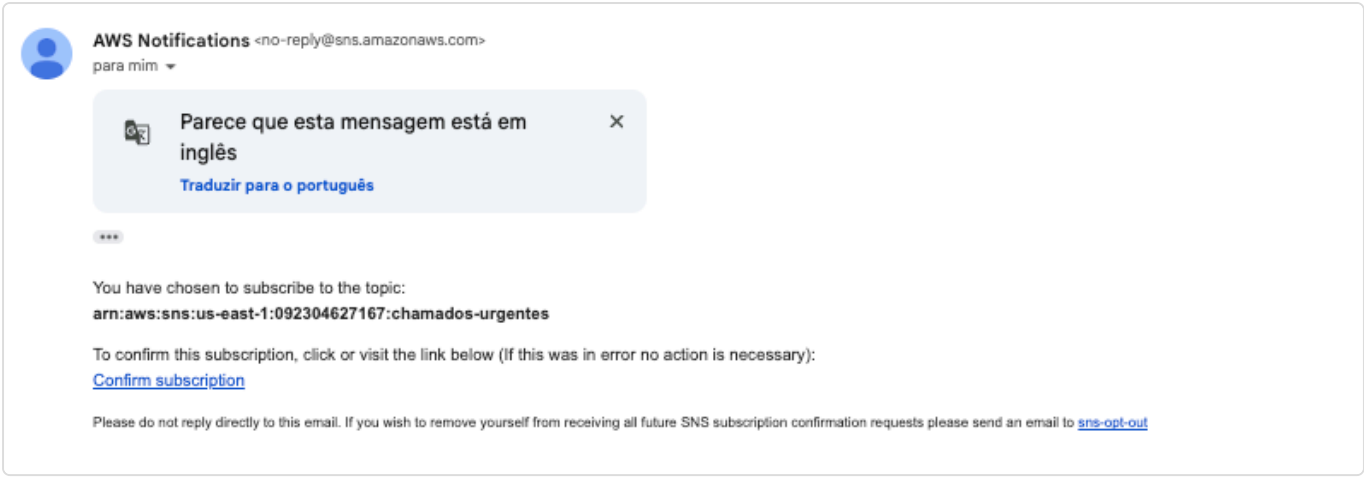
Tópico SNS e assinatura por e-mail



E04 · Tópico chamados-urgentes (SNS)



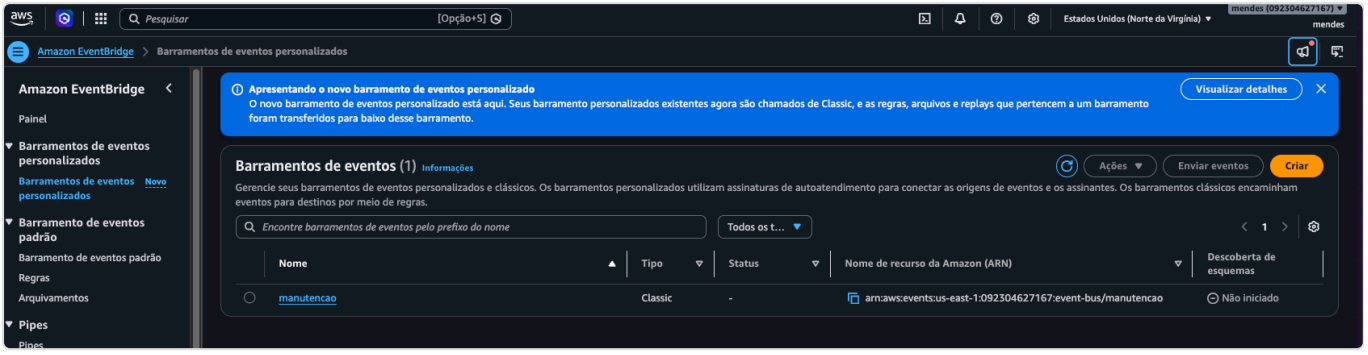
E04 · Assinatura de e-mail com status Confirmado, vinculada ao tópico chamados-urgentes



E04e · E-mail de confirmação enviado pelo SNS (AWS Notifications)

A assinatura só vale depois do clique em *Confirm subscription*; o status Confirmado prova que o alerta urgente pode chegar.

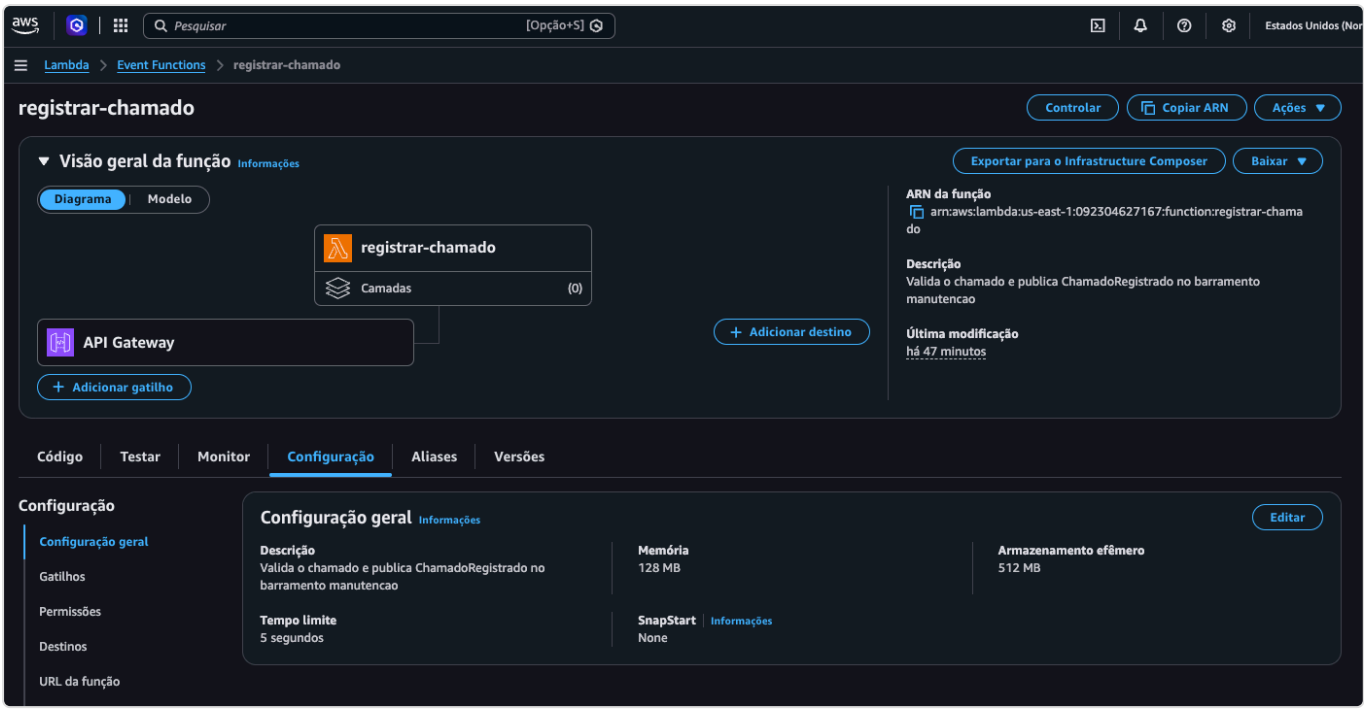
Barramento customizado



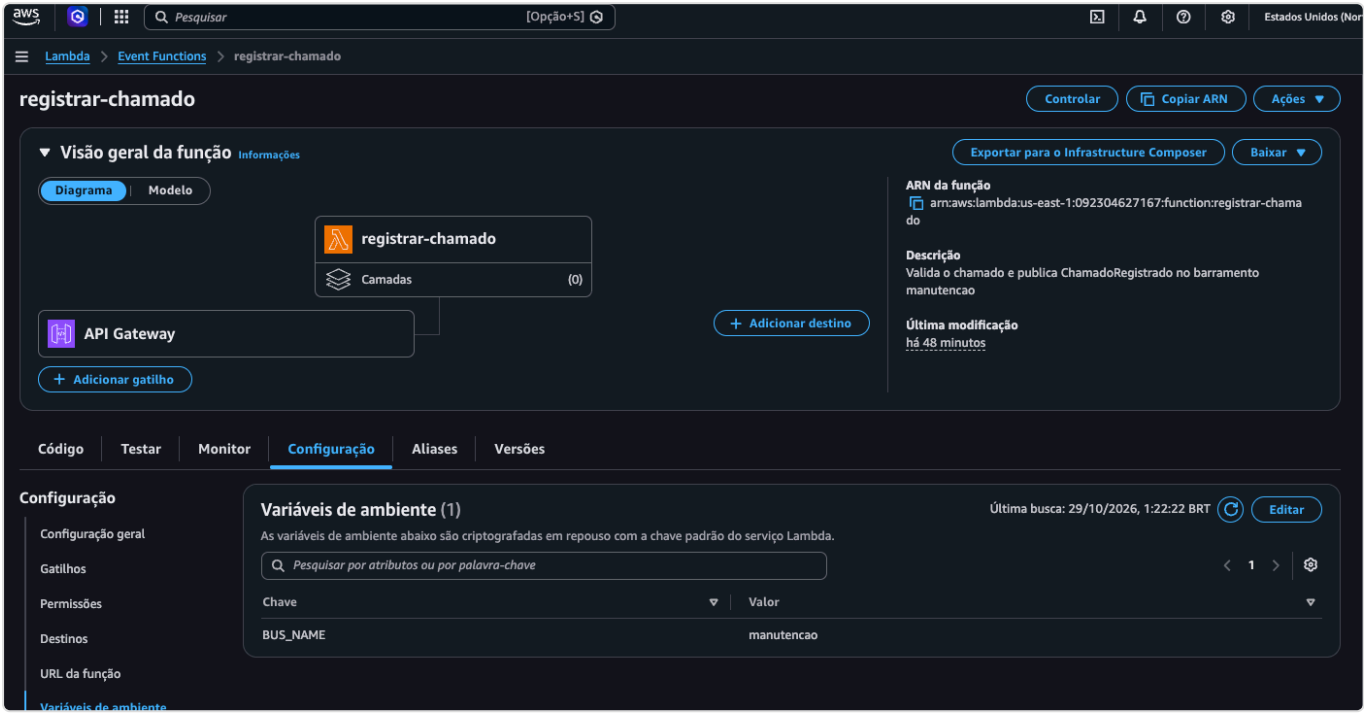
E05 · Barramento de eventos manutencao

O barramento `manutencao` é separado do barramento padrão, então os eventos da central ficam isolados dos eventos da conta.

Configuração da função registrar-chamado



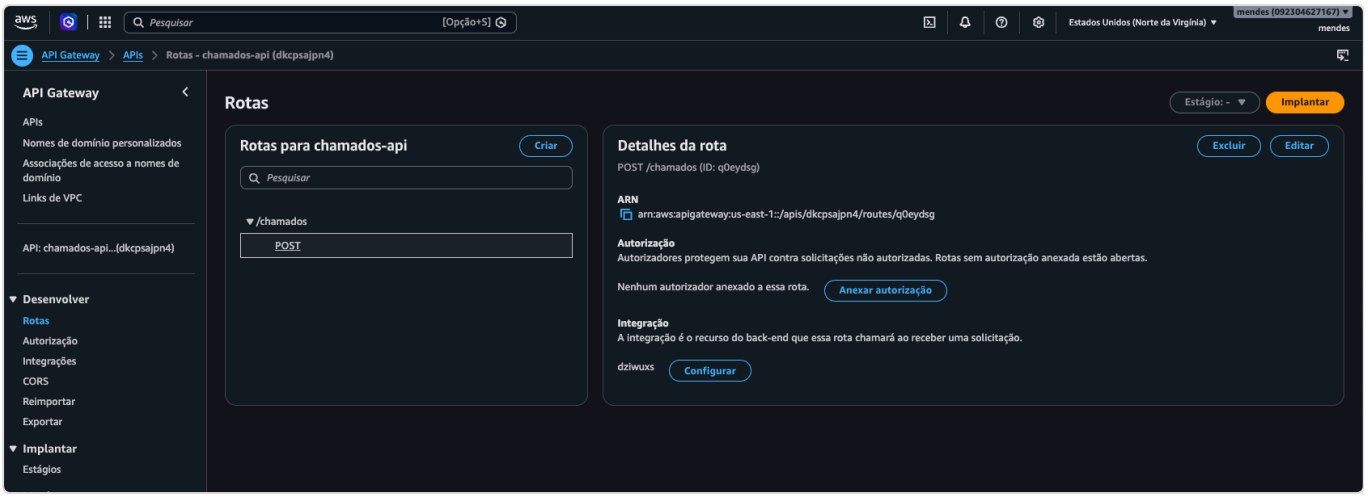
E06a · Configuração geral: tempo limite de 5 segundos e 128 MB de memória



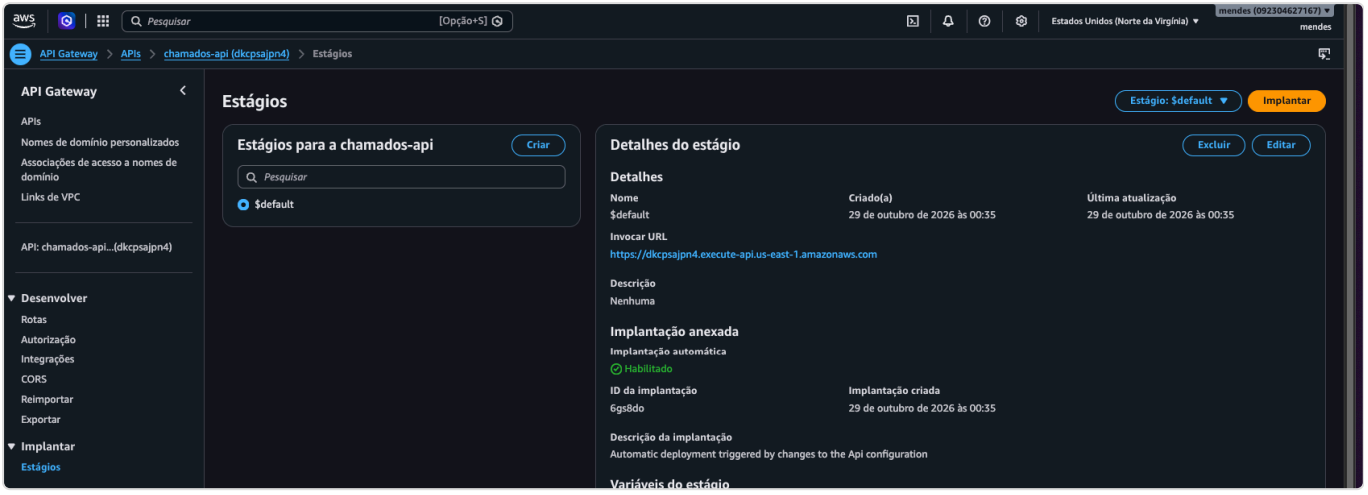
E06b · Variável de ambiente BUS_NAME = manutencao

O nome do barramento vem de variável de ambiente, não do código.

A API



E07a · API chamados-api: rota POST /chamados



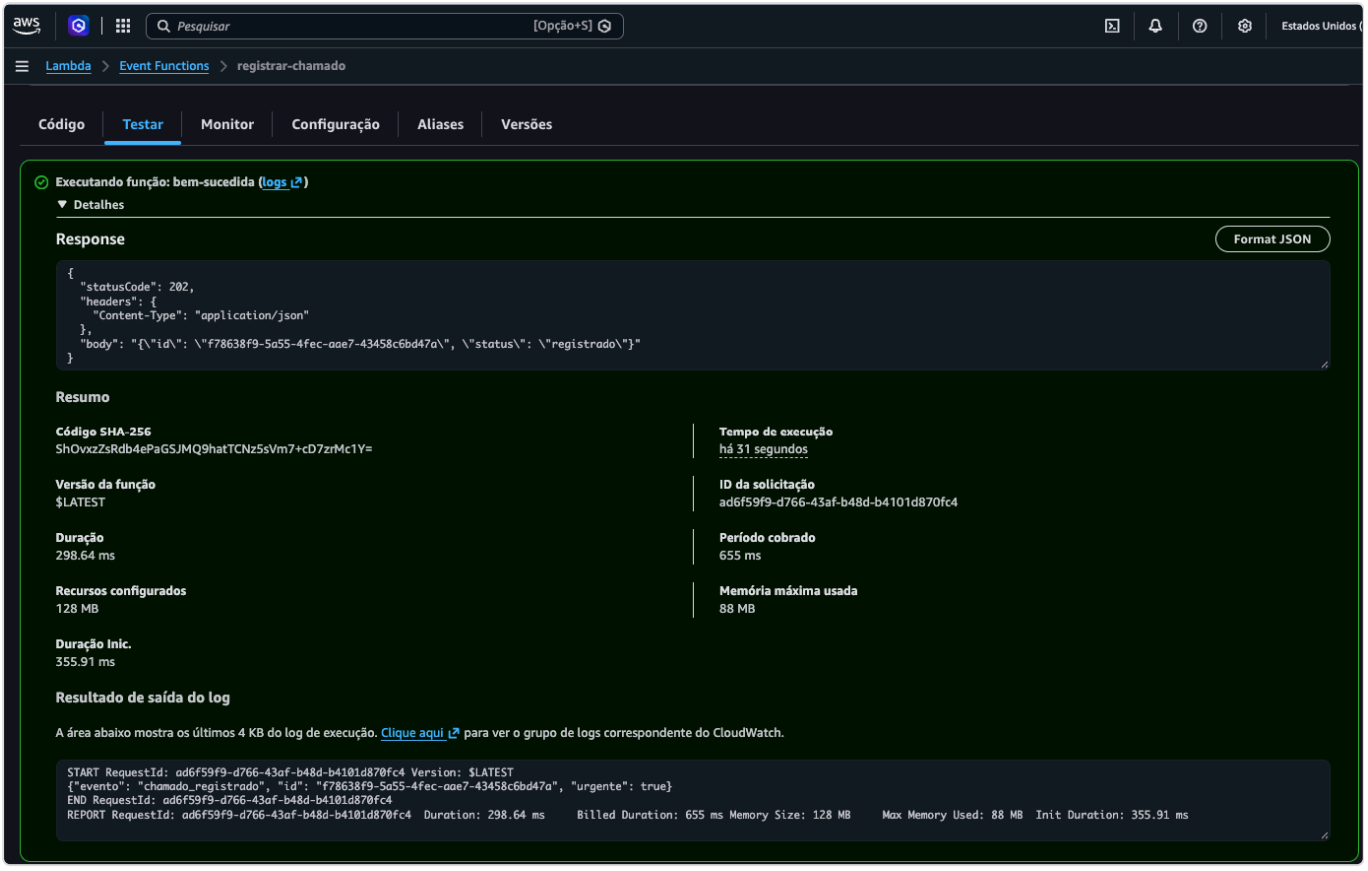
E07b · Estágio $default com implantação automática e a URL de invocação

O único ponto de entrada é `POST /chamados`, integrado à função `registrar-chamado`.

3. Testes

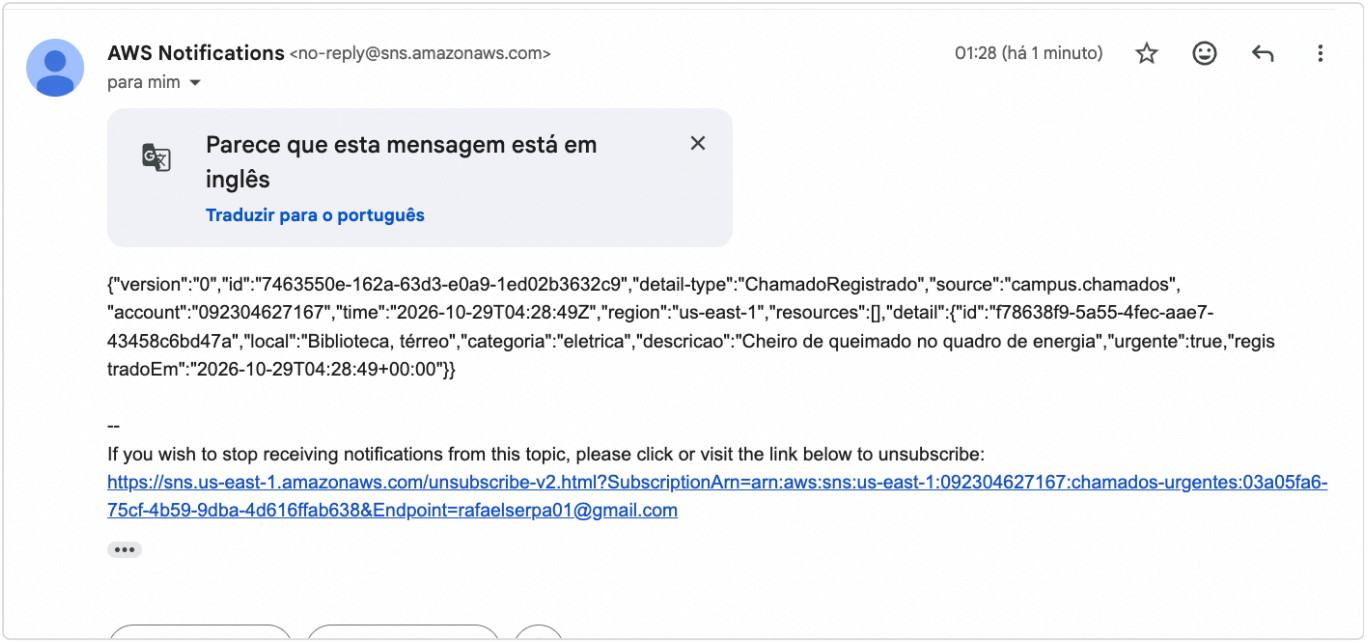
Os testes seguem a ordem do roteiro: primeiro a função isolada (Passo 6), depois o fluxo completo pela API (Passo 9) e, por fim, as regras do barramento (Passo 10).

3.1 Passo 6: a função isolada, pelo console



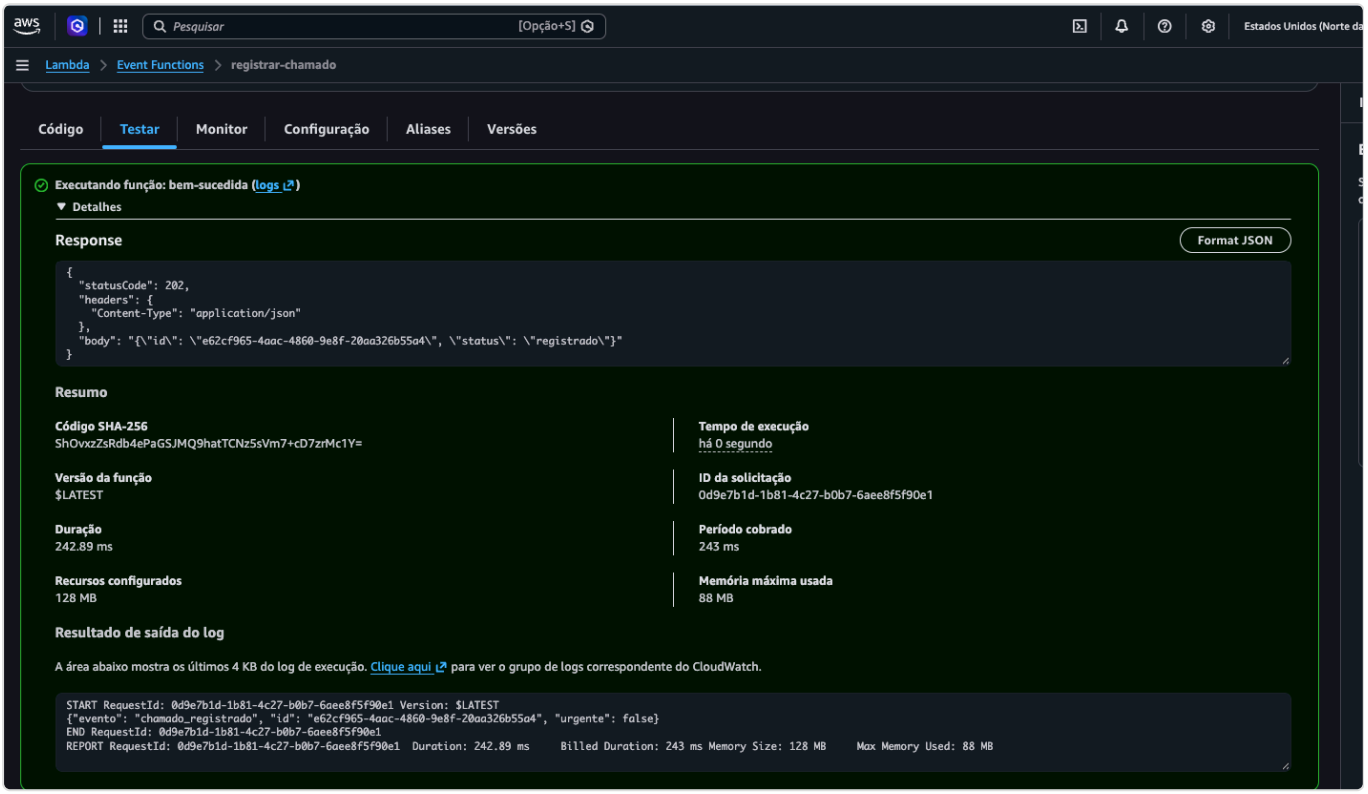
E08a · Teste urgente: execução bem-sucedida e resposta statusCode 202 com o id do chamado

O teste urgente (`urgente: true`, local Biblioteca) termina com *bem-sucedida* e devolve `statusCode: 202` e o `id` `f78638f9-5a55-4fec-aae7-43458c6bd47a`. São duas camadas: a plataforma Lambda diz que a execução terminou; o 202 é o código que a própria função devolve.



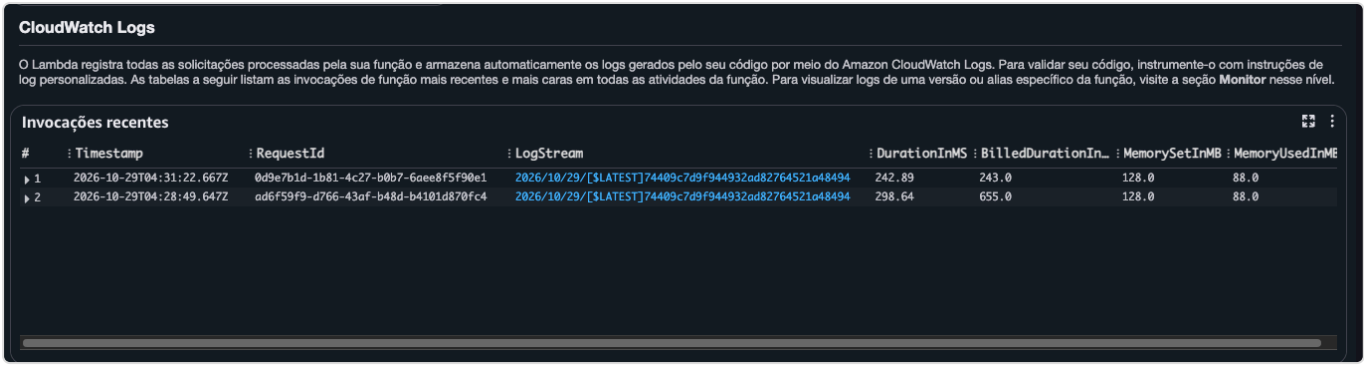
E08e · E-mail de alerta recebido (Biblioteca, térreo, 01:28)

O alerta chegou às 01:28, com o evento `ChamadoRegistrado` e a descrição "Cheiro de queimado no quadro de energia".



E08b · Teste comum: statusCode 202 com outro id

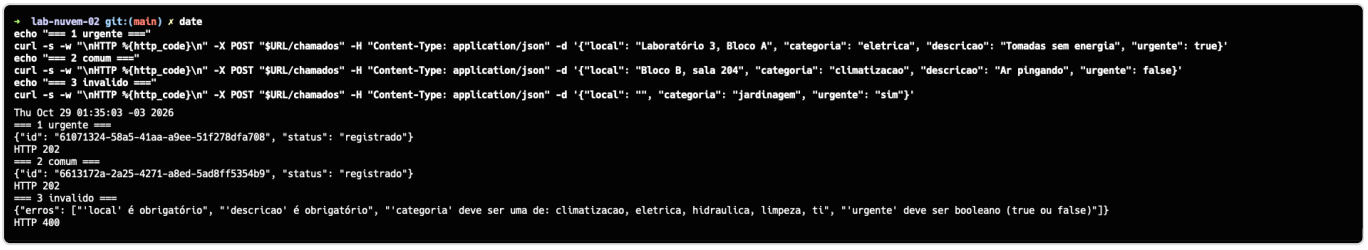
O chamado comum (`urgente: false`) também é aceito (202), com o `id` `e62cf965-4aac-4860-9e8f-20aa326b55a4`, e não gerou e-mail.



E08c · Monitor: as duas invocações recentes de registrar-chamado no CloudWatch Logs

A função registrou uma linha `chamado_registrado` por invocação (04:28:49Z e 04:31:22Z), visível também na saída de cada teste acima.

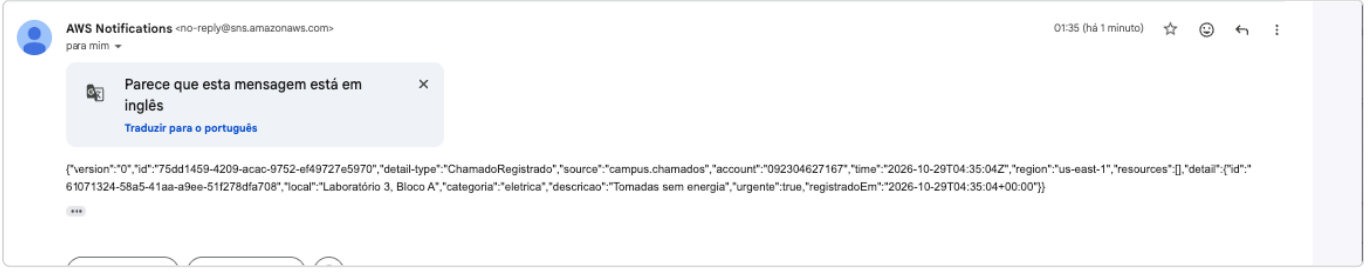
3.2 Passo 9: o fluxo completo pela API



E09 · Três testes com curl em 29/10/2026, 01:35:03: 202, 202 e 400

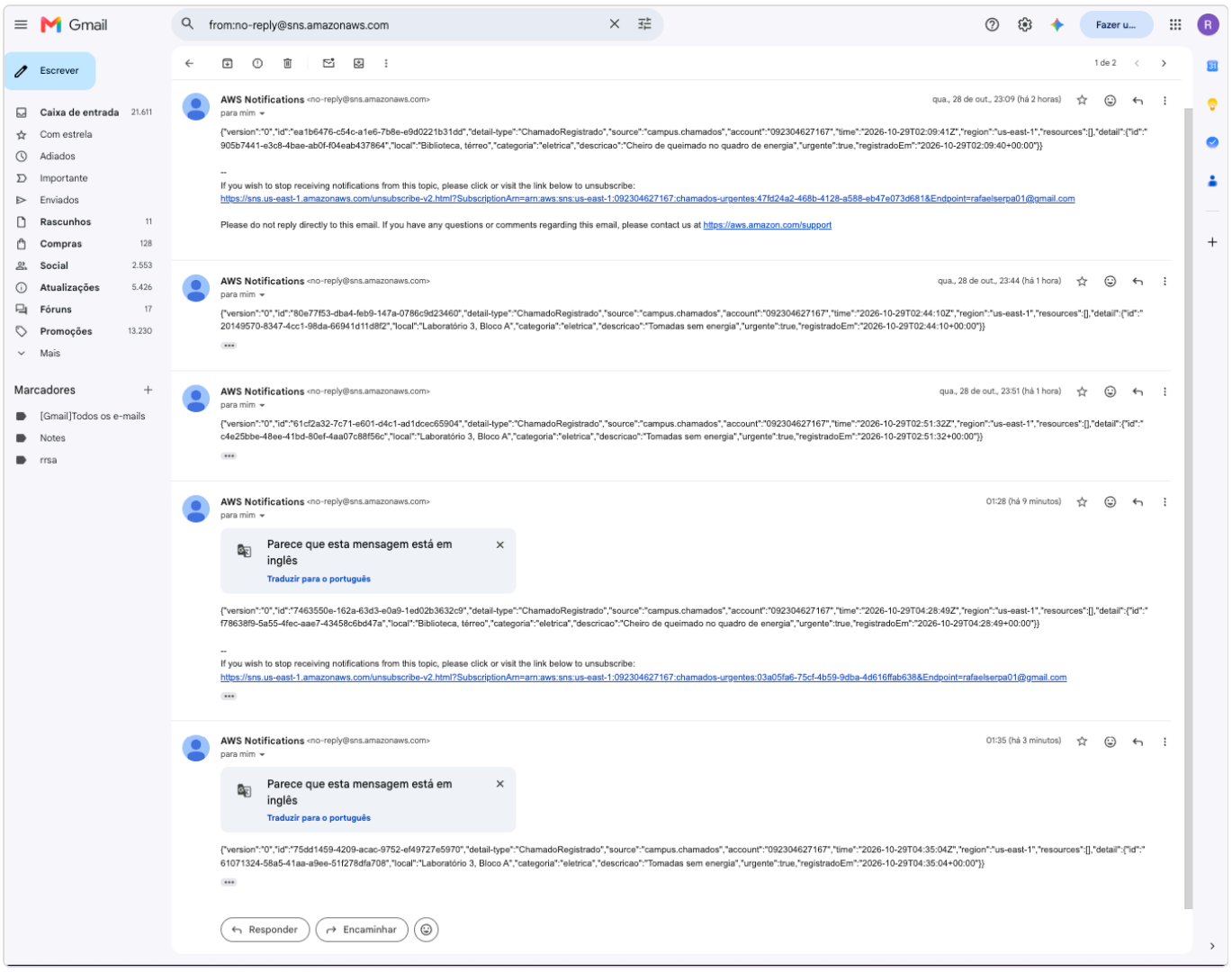
Os três testes foram disparados juntos, às 01:35:03 (-03):

Teste	Entrada	Resposta
1 urgente	Laboratório 3, Bloco A, elétrica, `urgente: true`	HTTP 202, id `61071324-58a5-41aa-a9ee-51f278dfa708`
2 comum	Bloco B, sala 204, climatização, `urgente: false`	HTTP 202, id `6613172a-2a25-4271-a8ed-5ad8ff5354b9`
3 inválido	`local` vazio, categoria jardinagem, `urgente: "sim"`	HTTP 400 com quatro erros de validação



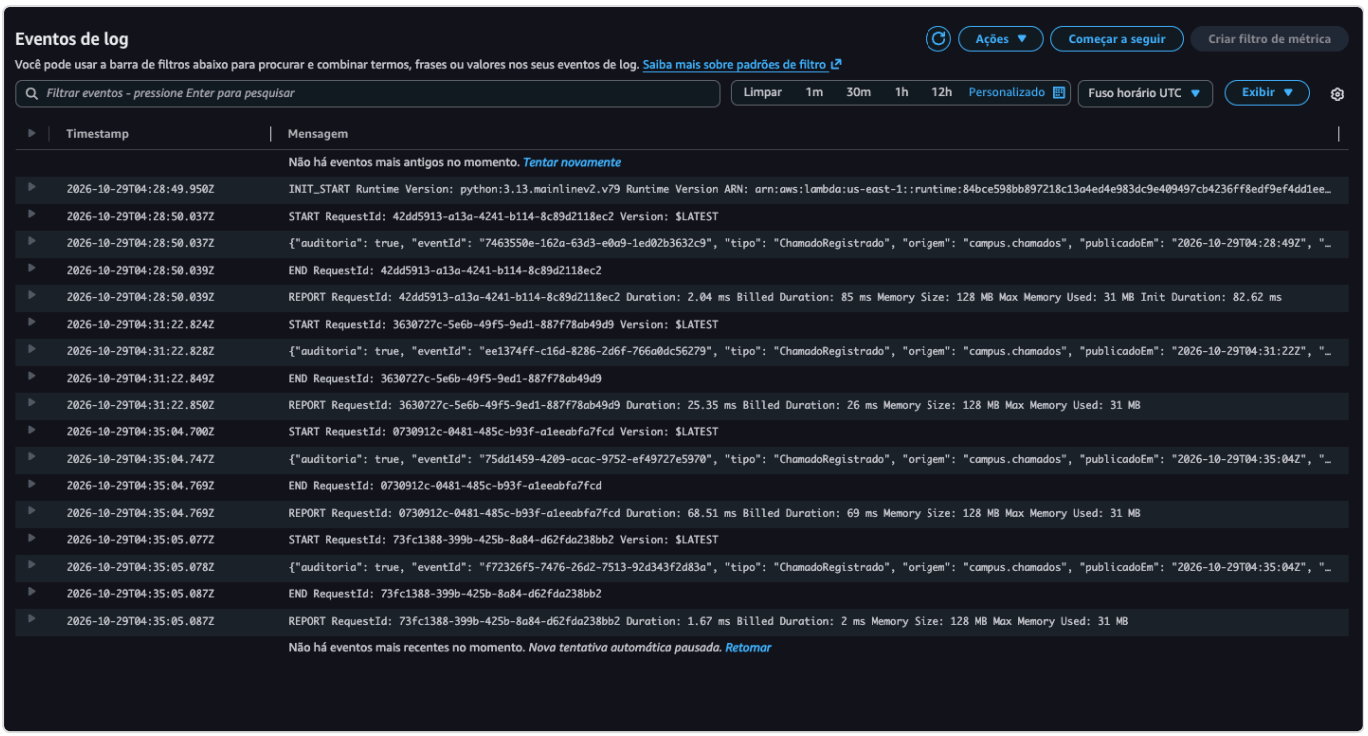
E10 · E-mail do teste urgente (Laboratório 3, Bloco A, 01:35)

O e-mail das 01:35 ("Tomadas sem energia") bate com o horário do teste 1.



E10b · Caixa de entrada filtrada por remetente SNS: só os e-mails dos chamados urgentes

Depois do teste de 01:35 não chegou nenhum outro e-mail: o chamado comum e o inválido não geraram alerta. Os três e-mails de 28/10 (23:09, 23:44 e 23:51) são de testes de uma implantação anterior; os de 01:28 e 01:35 são os desta entrega.

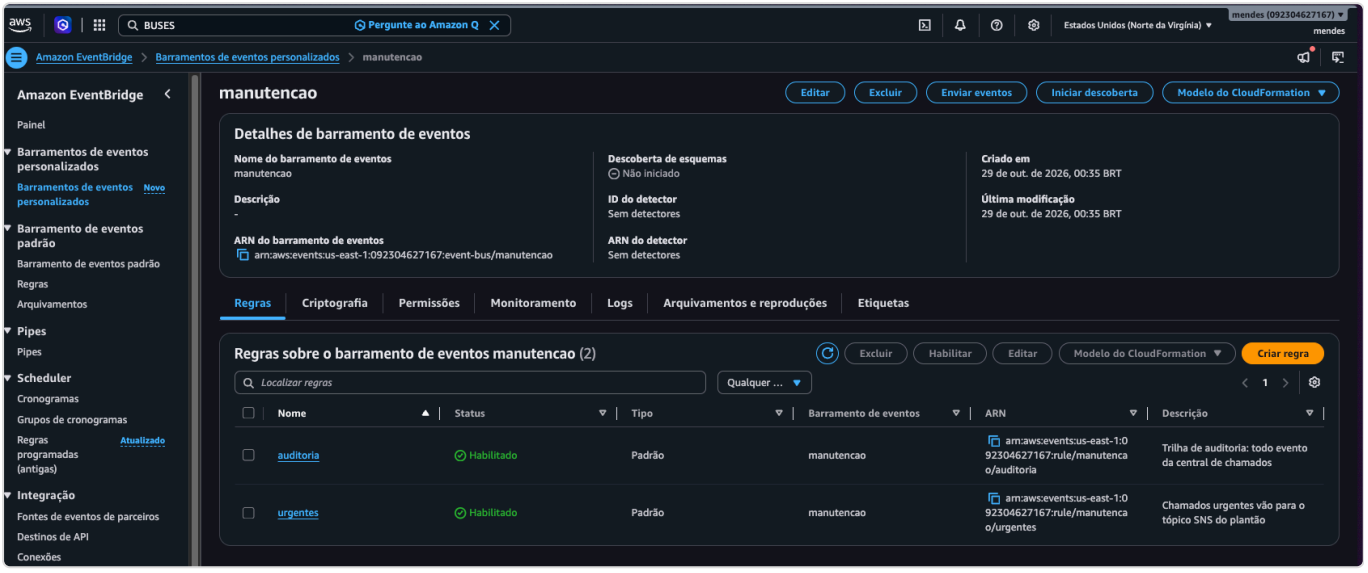


E11 · Log de auditoria (auditar-chamado): quatro eventos com "auditoria": true

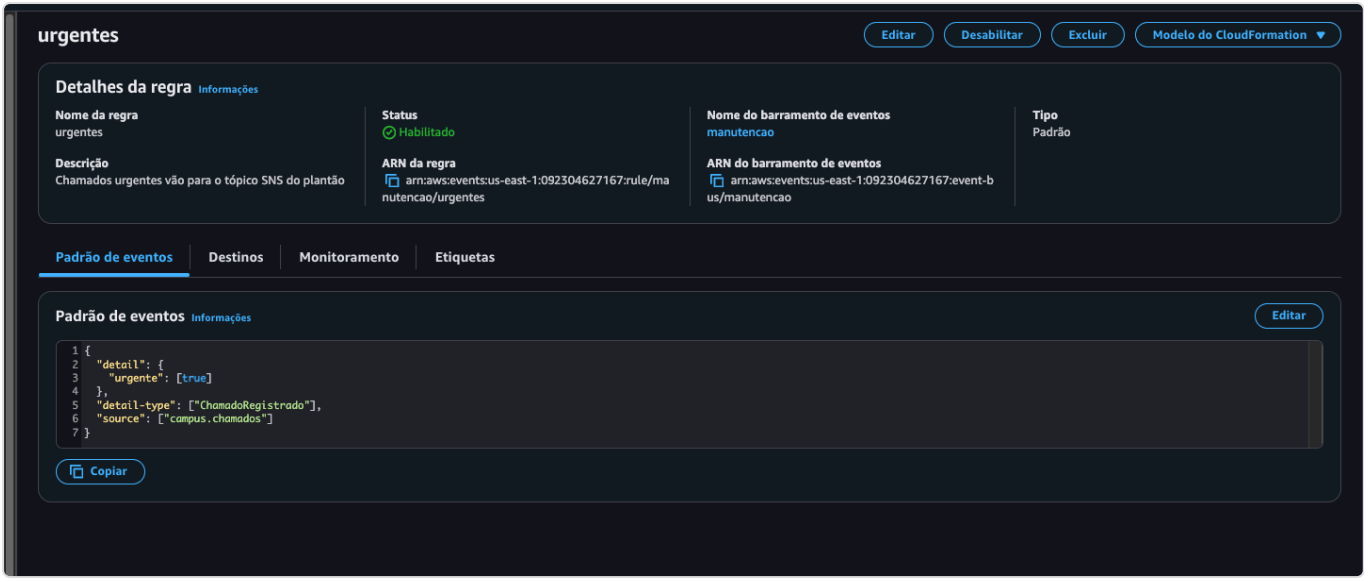
O log group `/aws/lambda/auditar-chamado` registrou quatro eventos auditados, em 04:28:49Z e 04:31:22Z (Passo 6) e 04:35:04Z e 04:35:05Z (Passo 9, urgente e comum). O `eventId` de 04:28:49Z (`7463550e-…`) e o de 04:35:04Z (`75dd1459-…`) são os mesmos dos dois e-mails recebidos, o que liga o alerta à trilha. O teste inválido não aparece, porque a validação barrou antes de publicar.

A própria captura E11 mostra a janela de tempo completa dos testes com apenas esses quatro eventos, e nenhum deles contém `jardinagem`. Como o chamado inválido foi rejeitado com 400 antes do `PutEvents` (E09), nenhum evento foi publicado para ele.

3.3 Regras do barramento e roteamento

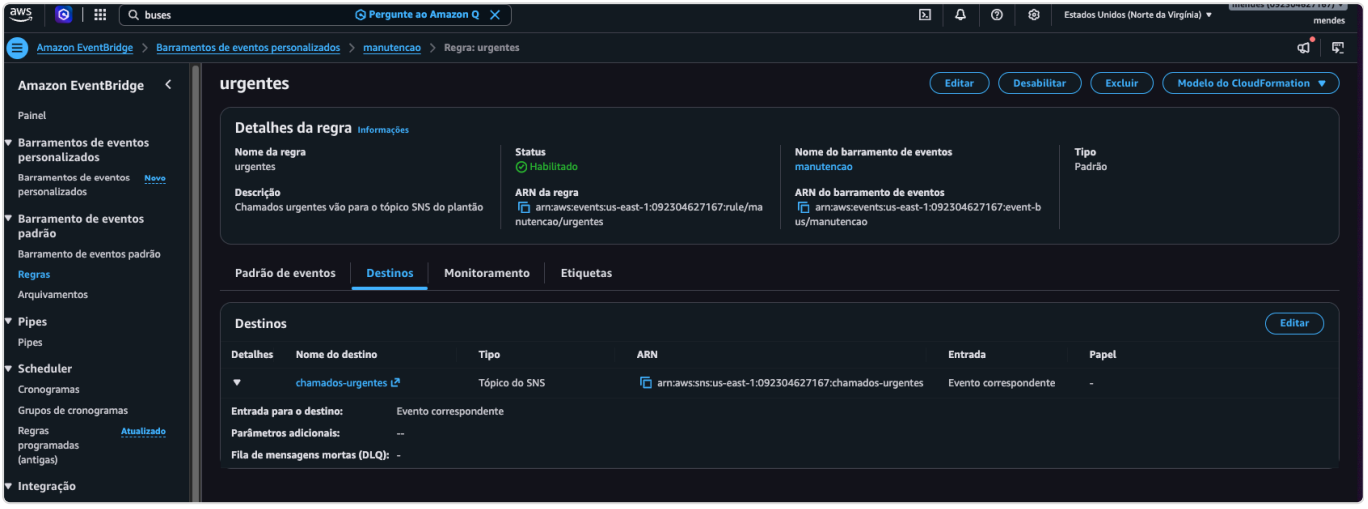


E12a · Barramento manutencao com as duas regras habilitadas: auditoria e urgentes

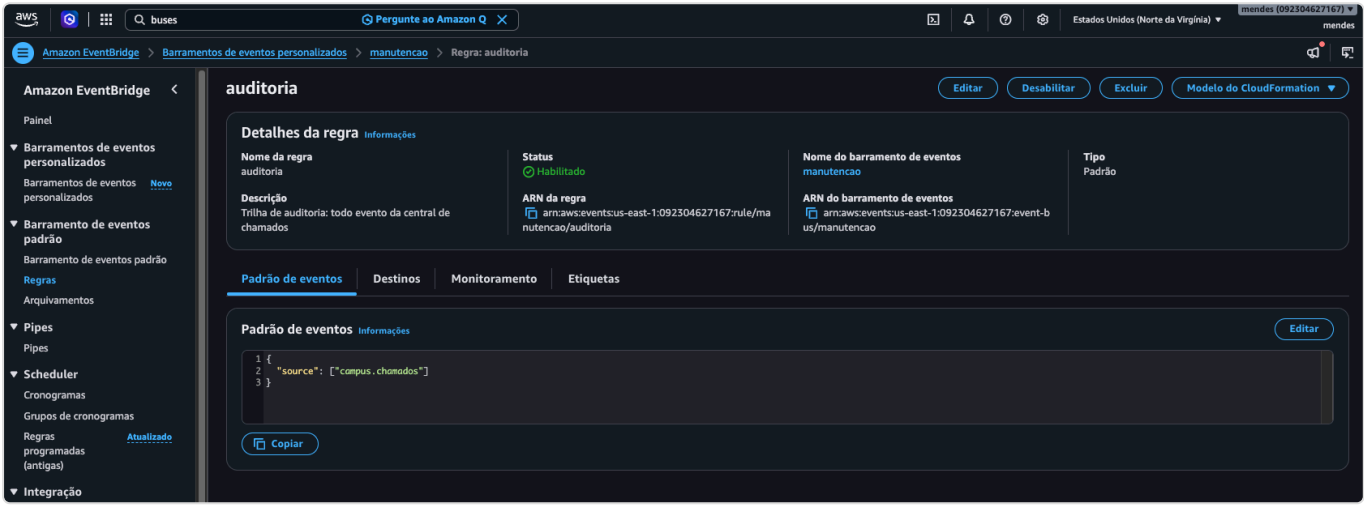


E12b1 · Regra urgentes: padrão de eventos com detail.urgente = [true]

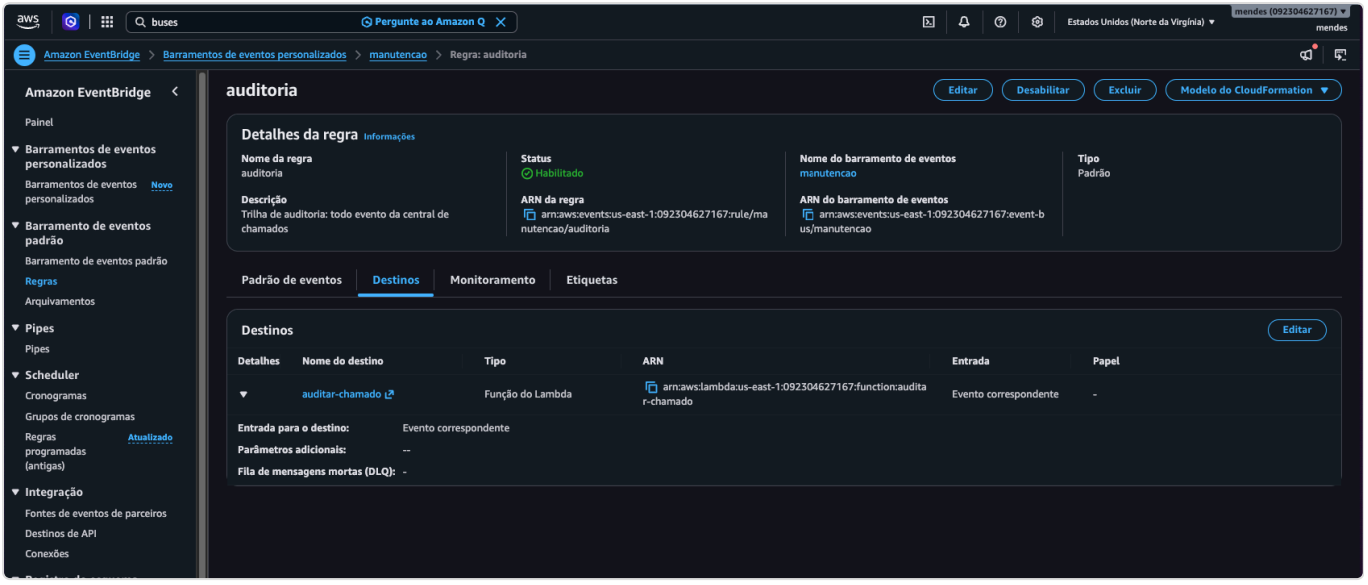
A regra `urgentes` casa só eventos de `campus.chamados`, tipo `ChamadoRegistrado`, com `detail.urgente` igual a `true`.



E12b2 · Regra urgentes: destino tópico SNS chamados-urgentes

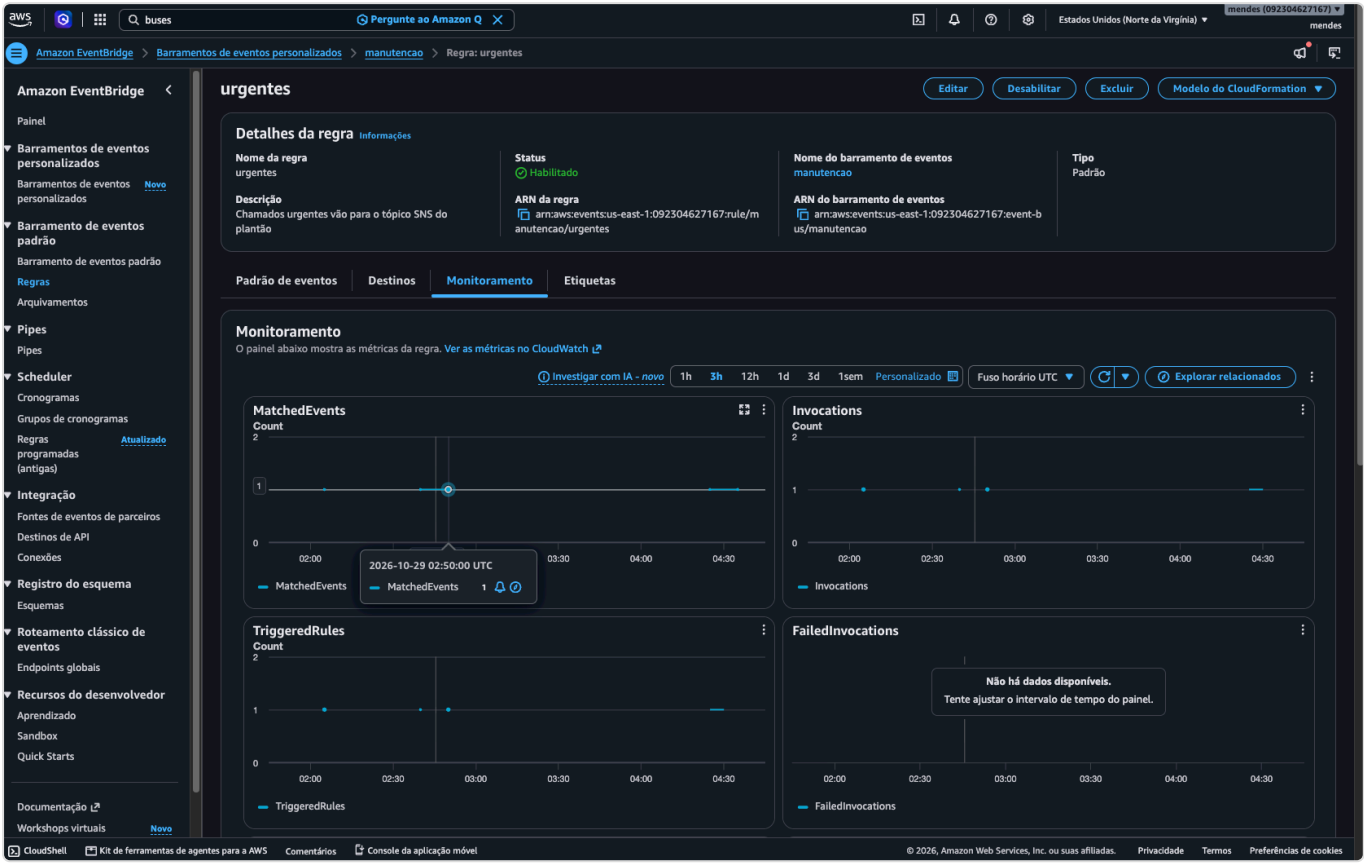


E12c1 · Regra auditoria: padrão largo, só source = campus.chamados



E12c2 · Regra auditoria: destino função Lambda auditar-chamado

A regra `auditoria` filtra só pela origem, então recebe todo evento da central, urgente ou não.



E13a · Métricas da regra urgentes (3 h)

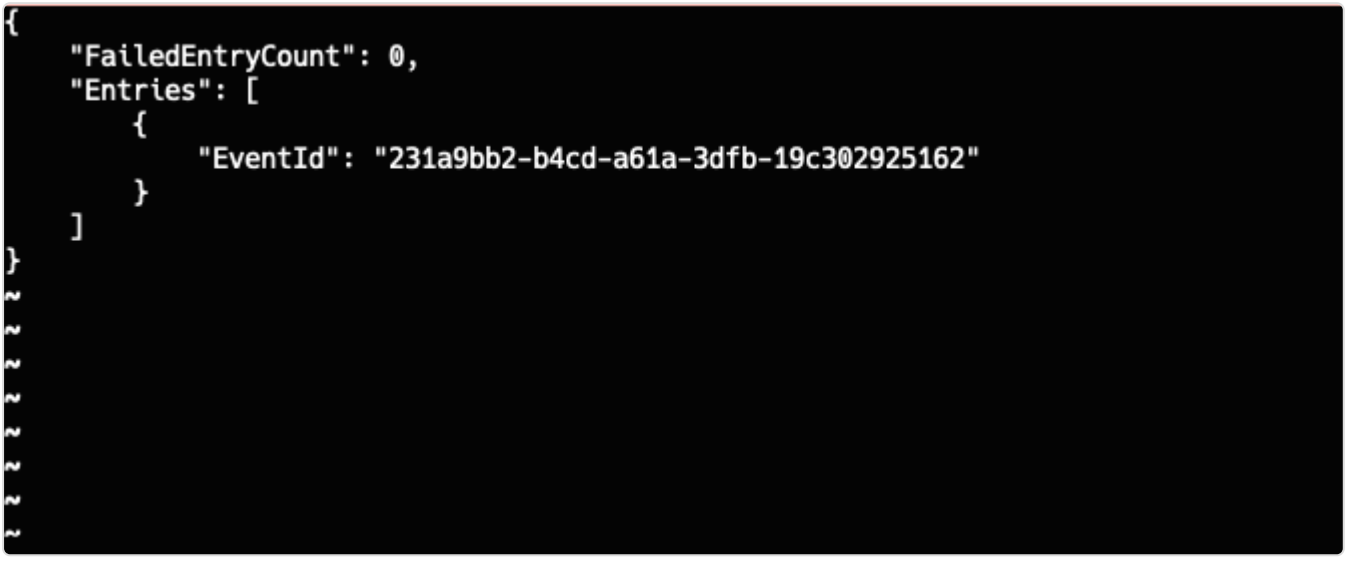


E13b · Métricas da regra auditoria (3 h): mais acionamentos que a urgentes

As métricas mostram a diferença de volume: a auditoria recebe tudo e a regra de urgentes só um subconjunto. Os gráficos de 3 h também incluem tráfego da implantação anterior (pontos por volta de 02:00 UTC).

Passo 10.2: quatro eventos publicados direto no barramento

Para provar o casamento tipado do padrão `urgente: [true]`, quatro eventos foram publicados com `aws events put-events` no barramento `manutencao`, cada um com uma descrição marcada (`TESTE-A` a `TESTE-D`).



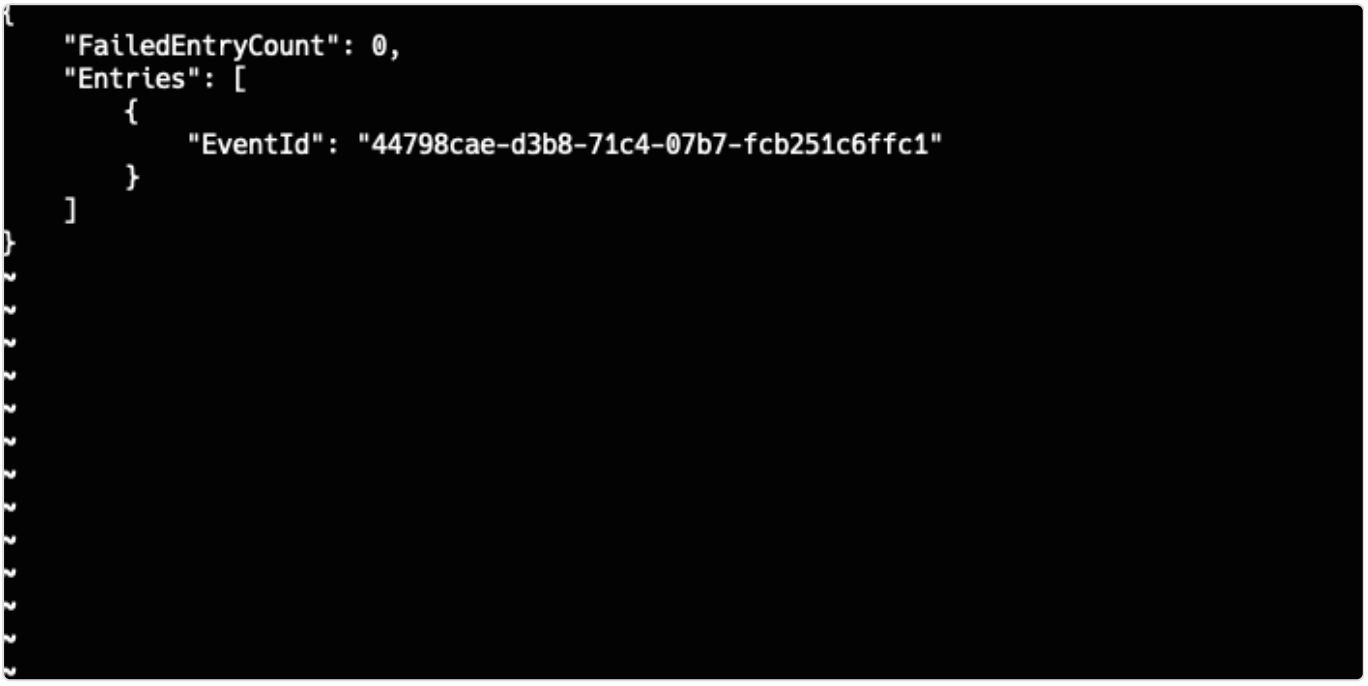
E14a · Evento A enviado: FailedEntryCount 0



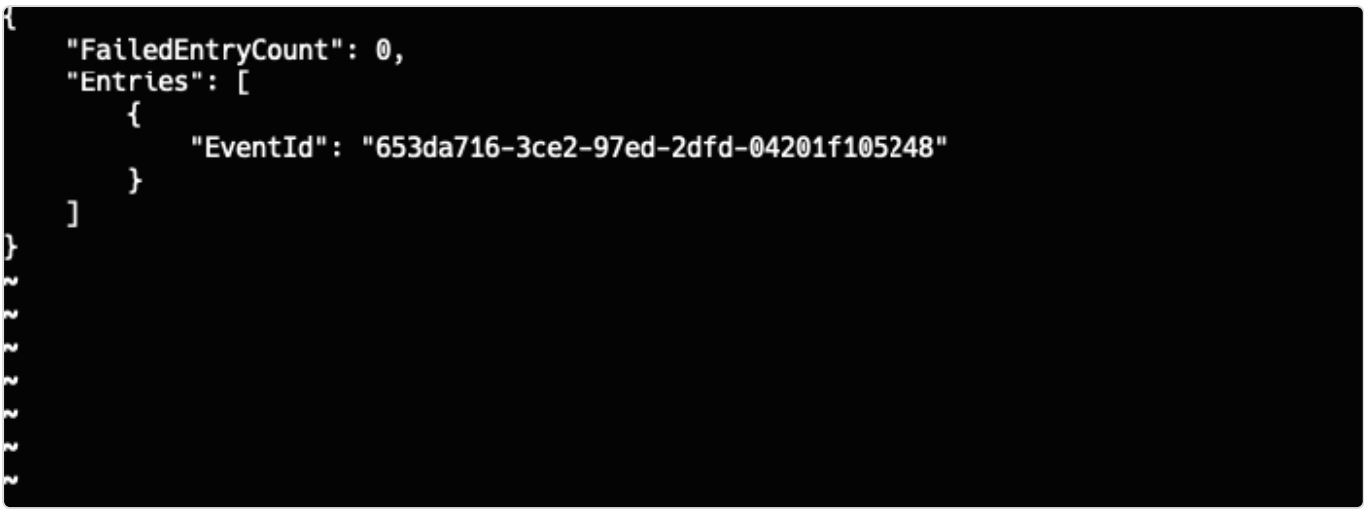
E14a · E-mail do evento A (TESTE-A urgente true, 01:57)



E14b · Evento B enviado: FailedEntryCount 0



E14c · Evento C enviado: FailedEntryCount 0



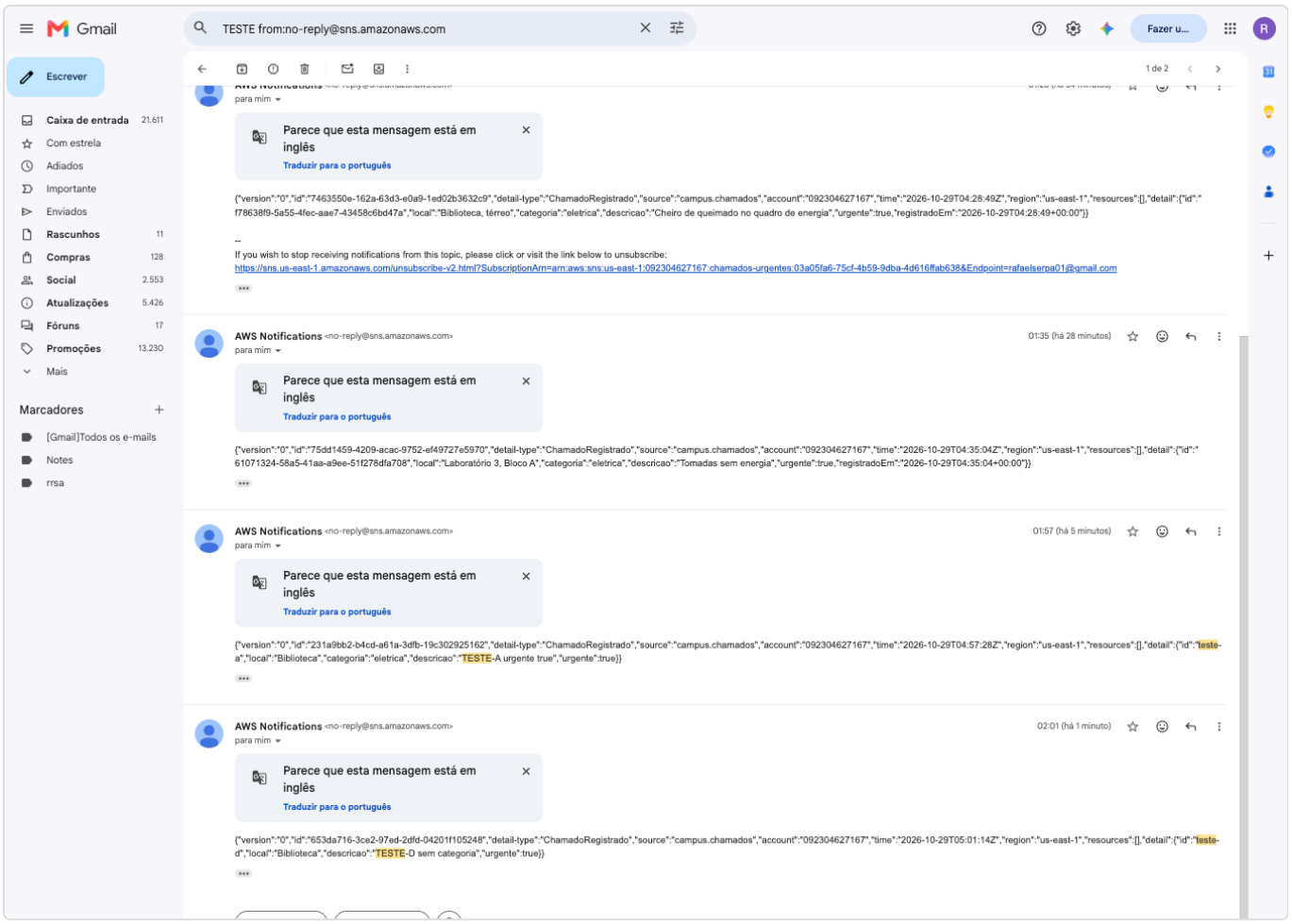
E14d · Evento D enviado: FailedEntryCount 0



E14d · E-mail do evento D (TESTE-D sem categoria, 02:01)



E14e · Log de auditoria filtrado por TESTE: os quatro eventos A, B, C e D



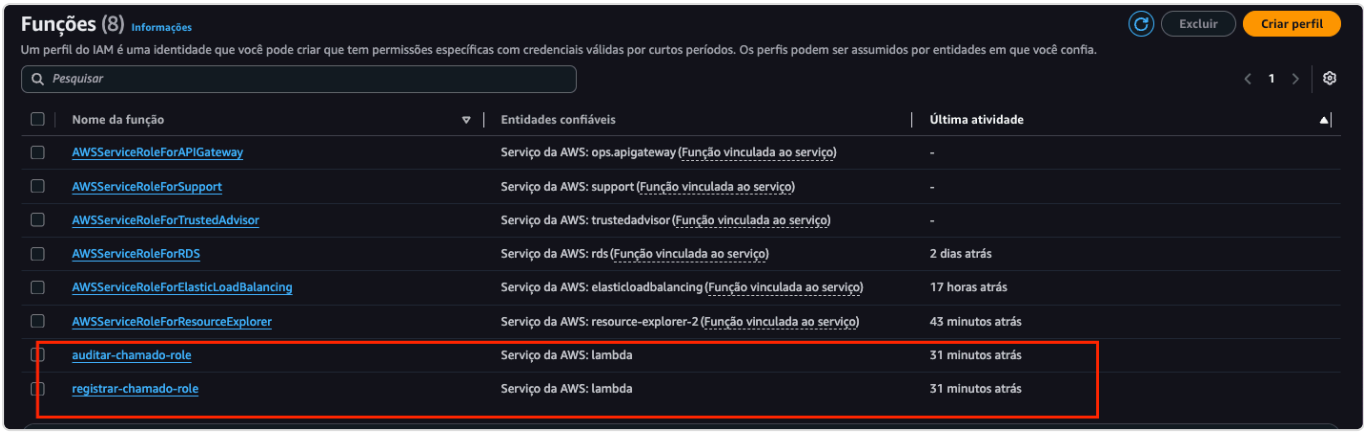
E14f · Caixa de entrada filtrada por TESTE: só os eventos A e D geraram e-mail

Evento	`urgente`	E-mail	Auditoria	Por quê
A	`true` (booleano)	Sim	Sim	O padrão pede `true` e o evento tem `true`.
B	`false`	Não	Sim	`false` não está na lista de valores aceitos.
C	`"true"` (texto)	Não	Sim	O casamento é tipado: o texto `"true"` não é o booleano `true`.
D	`true`, sem `categoria`	Sim	Sim	`categoria` não aparece no padrão, então não é filtrada.

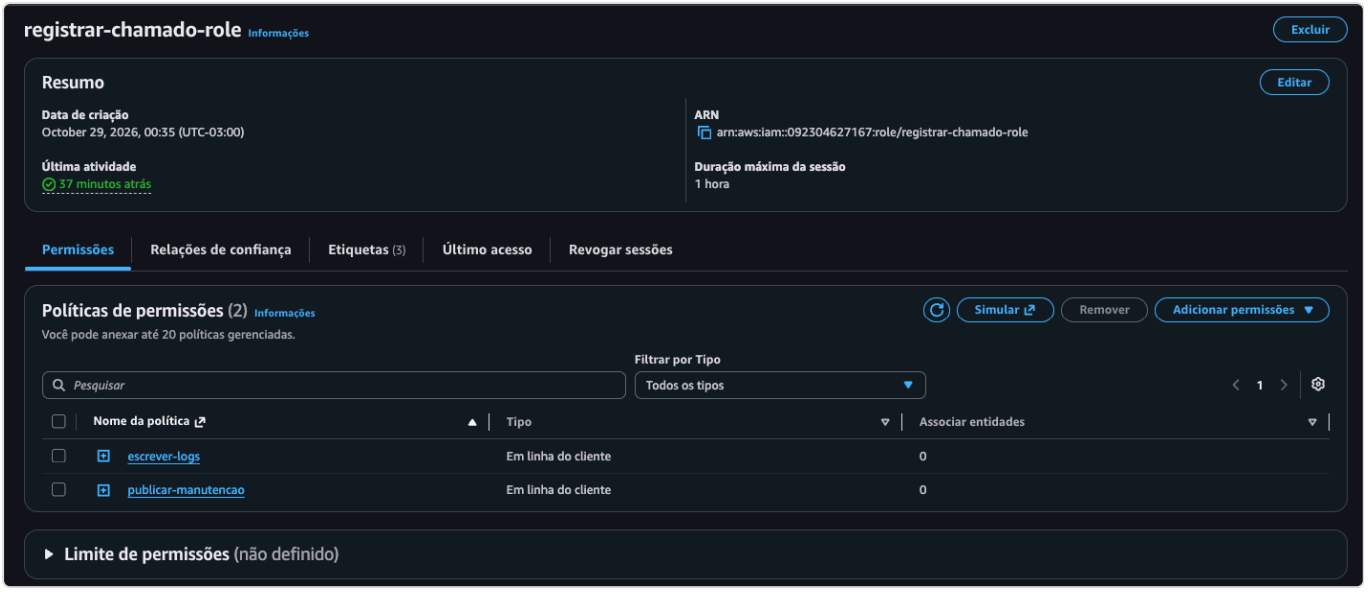
Os quatro foram auditados porque o padrão da regra `auditoria` só olha o `source`.

4. Política IAM de menor privilégio

A função `registrar-chamado` só pode publicar eventos no barramento `manutencao`, e mais nada.



E15 · Roles IAM: auditar-chamado-role e registrar-chamado-role, uma por função



E15a · registrar-chamado-role: duas políticas em linha, escrever-logs e publicar-manutencao

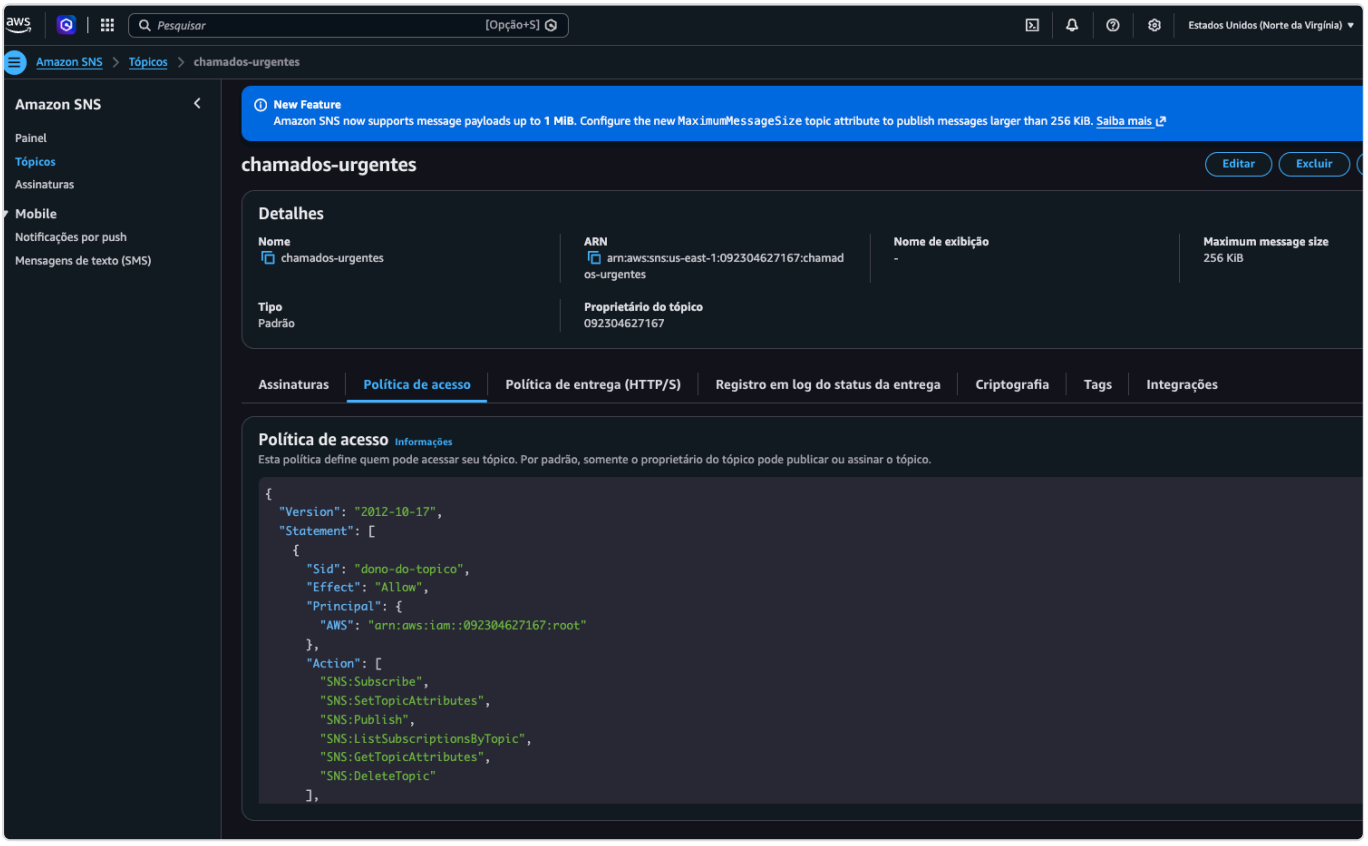


E15b · Política publicar-manutencao: uma ação e um recurso

Política `publicar-manutencao` (ID da conta trocado por `123456789012`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "publicarSomenteNoBarramentoManutencao",
      "Effect": "Allow",
      "Action": "events:PutEvents",
      "Resource": "arn:aws:events:us-east-1:123456789012:event-bus/manutencao"
    }
  ]
}
```

Uma única ação (`events:PutEvents`) e um único recurso, o ARN do barramento `manutencao`, sem `*`. Publicar no barramento padrão ou em qualquer outro é negado por omissão.



E17a · Política de acesso do tópico chamados-urgentes (SNS), lado do recurso

A captura acima mostra o primeiro statement da política. O segundo, reproduzido abaixo a partir do código Terraform, é o que autoriza o EventBridge:

```
{
  "Sid": "eventbridge-publica",
  "Effect": "Allow",
  "Principal": { "Service": "events.amazonaws.com" },
  "Action": "sns:Publish",
  "Resource": "arn:aws:sns:us-east-1:123456789012:chamados-urgentes",
  "Condition": {
    "ArnEquals": {
      "aws:SourceArn": "arn:aws:events:us-east-1:123456789012:rule/manutencao/urgentes"
    }
  }
}
```

São duas metades diferentes da permissão. A policy da role (acima) diz o que a **função** pode fazer; esta diz **quem** pode publicar **no tópico**. A condição `ArnEquals` em `aws:SourceArn` restringe não só ao serviço EventBridge, mas à regra `urgentes` especificamente: outra regra da mesma conta não conseguiria publicar neste tópico.

Demais políticas da arquitetura

As duas funções usam a mesma **trust policy**, que define quem pode assumir a role:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Principal": { "Service": "lambda.amazonaws.com" }
    }
  ]
}
```

E cada uma tem uma policy `escrever-logs` apontando para o próprio log group:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "escreverLogsDaPropriaFuncao",
      "Effect": "Allow",
      "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
      "Resource": "arn:aws:logs:us-east-1:123456789012:log-group:/aws/lambda/registrar-chamado:*"
    }
  ]
}
```

A role `auditar-chamado-role` tem a mesma policy, trocando o log group por `/aws/lambda/auditar-chamado`. Nenhuma das duas tem `logs:CreateLogGroup`: o log group é criado pelo Terraform, então a função não precisa do direito de criar log groups na conta.

A função `auditar-chamado` também tem uma permissão do lado do recurso, `AllowExecutionFromEventBridge`, que deixa só a regra `auditoria` invocá-la — ela aparece sendo removida no destroy (seção 6). O mesmo vale para a API: `AllowExecutionFromAPIGateway` autoriza a `chamados-api` a invocar `registrar-chamado`.

5. Respostas às sete perguntas

1. Quem chamou a função no Passo 6 e no Passo 9? A função percebeu a diferença?

No Passo 6 quem chamou foi o **console da Lambda**, pela aba Testar, que faz um `Invoke` direto no serviço Lambda com as credenciais do usuário logado. No Passo 9 quem chamou foi o **API Gateway**, por causa da integração `AWS_PROXY` da rota `POST /chamados`, usando a permissão de recurso `AllowExecutionFromAPIGateway`.

A função **não percebeu a diferença**, porque recebeu o mesmo formato de evento nos dois casos: um JSON com a chave `body` guardando o corpo da requisição como texto. No Passo 6 montamos esse envelope à mão; no Passo 9 o API Gateway monta sozinho (formato de payload 2.0). O handler só lê `event["body"]` e faz `json.loads`, então o chamador é indiferente. A prova está nas evidências: o teste do console devolveu `statusCode: 202` com `id` e `status` (E08a), exatamente o mesmo corpo que o `curl` recebeu (E09).

O que muda é **quem traduz a resposta**. No Passo 6 vimos o dicionário cru `{statusCode, headers, body}` na tela do console; no Passo 9 o API Gateway converteu esse dicionário numa resposta HTTP de verdade, com o 202 na linha de status.

2. Para onde foi o evento do teste comum no Passo 6? E no Passo 9, depois da regra auditoria?

Depende de quais regras existem no barramento no momento da publicação.

No roteiro original, a regra `auditoria` só nasce no Passo 8. Então o evento comum do Passo 6 seria publicado no barramento, não casaria com nenhuma regra (a `urgentes` exige `urgente: true`) e seria **descartado**. O EventBridge não é fila: evento sem regra correspondente não fica guardado esperando consumidor, ele simplesmente desaparece.

No nosso ambiente, a regra `auditoria` já existia quando rodamos o Passo 6, porque todo o `terraform apply` foi feito de uma vez. Então o evento comum casou com a `auditoria`, que filtra só por `source`, e foi entregue à função `auditar-chamado` — é o registro de 04:31:22Z em E11. Continuou sem e-mail, porque não casou com a `urgentes`.

No Passo 9 o comportamento foi o mesmo: o chamado comum (Bloco B, "Ar pingando") foi auditado às 04:35:05Z (E11) e não gerou e-mail (E10b).

3. Vantagem e desvantagem de responder 202 antes de o e-mail sair

Vantagem: a resposta é rápida e previsível. O cliente espera só pela validação e pelo `PutEvents` — a função levou 298 ms na primeira invocação e 243 ms na segunda (E08a e E08b). Esse tempo não depende do SNS, do servidor de e-mail nem de quantos consumidores existem. Se amanhã entrarem mais três consumidores, a latência de quem chama a API não muda, e uma lentidão do SNS não vira erro na tela do usuário.

Desvantagem: o 202 é a promessa de que o chamado foi **aceito**, não a confirmação de que alguém foi avisado. Se a regra estiver desabilitada, se a assinatura do SNS não estiver confirmada ou se a entrega falhar, o cliente já recebeu sucesso e nunca fica sabendo. Foi literalmente o que aconteceu com os eventos B e C do Passo 10.2: `FailedEntryCount: 0` na publicação e nenhum e-mail (E14b, E14c, E14f). Para fechar essa lacuna seria preciso observar métricas e logs, ou expor uma consulta do chamado pelo `id`.

4. Se o SNS estivesse fora do ar no teste urgente

A API responderia **202 do mesmo jeito**. A função `registrar-chamado` só conversa com o EventBridge; ela nem sabe que o SNS existe — a policy dela não tem nenhuma permissão de SNS (E15b). Enquanto o `PutEvents` tiver sucesso, o 202 sai.

Quem tentaria entregar de novo é o **EventBridge**. Ele faz retentativas automáticas na entrega ao alvo, com espera crescente entre as tentativas, por até 24 horas por padrão. Se mesmo assim não conseguir, o evento se perde, a menos que o alvo tenha uma fila de mensagens mortas (DLQ) configurada. Na nossa regra `urgentes` o campo DLQ está vazio (E12b2); em produção valeria configurar uma.

5. Por que publicar no EventBridge em vez de chamar SNS e auditoria direto?

O motivo principal é **desacoplamento**: a função `registrar-chamado` não conhece nenhum consumidor. Isso aparece na própria policy dela, que tem só `events:PutEvents` no barramento `manutencao` (E15b) — ela não tem permissão de publicar no SNS nem de invocar a Lambda de auditoria. Adicionar um consumidor é criar uma regra nova, sem tocar no código da função e sem novo deploy.

Outras vantagens:

- **Roteamento declarativo por conteúdo:** a regra `urgentes` decide pelo campo `urgente` do próprio evento (E12b1), sem nenhum `if` no código.
- **Falha isolada:** um consumidor quebrado não derruba o registro do chamado.
- **Observabilidade por rota:** cada regra tem métricas próprias (E13a e E13b).

Quando chamar direto seria aceitável: quando existe um consumidor só, conhecido e estável, e a resposta dele faz parte do caso de uso. Por exemplo, um endpoint que registra o chamado e precisa devolver, na mesma requisição, o número de protocolo vindo de um sistema legado: aí a indireção do barramento só acrescenta latência e peças. Também é aceitável num protótipo, onde a simplicidade vale mais que a extensibilidade.

6. Entrega pelo menos uma vez: o que quebra com duplicata e como evitar

No e-mail: o plantão recebe o mesmo alerta duas vezes. Em volume, o ruído faz as pessoas pararem de ler os alertas, o que é pior que o incômodo pontual.

Na auditoria: a trilha ganha duas linhas para o mesmo chamado. Se alguém contar linhas para medir volume, o número sai inflado. E se a auditoria evoluísse para gravar em banco ou abrir ordem de serviço, viraria chamado duplicado para a equipe de campo.

Como evitar: tornar o consumidor **idempotente**, usando uma chave estável. O evento já traz duas: o `id` do chamado, gerado pela função, e o `eventId` do EventBridge — os dois aparecem nos nossos logs (E11 e E14e). O consumidor guarda as chaves já processadas (por exemplo numa tabela DynamoDB com o `id` como chave primária e escrita condicional) e descarta o repetido. Para o e-mail, a mesma ideia: registrar que o alerta do chamado X já saiu antes de enviar. E escrever o consumidor de forma que reprocessar não cause efeito extra, usando `upsert` em vez de `insert`.

7. Custo mensal estimado para 10 mil chamados

Premissas: 10.000 chamados por mês, 10% urgentes (1.000 e-mails), região us-east-1, durações medidas nas próprias evidências (registrar ≈ 300 ms, auditar ≈ 50 ms, ambas com 128 MB), **sem**

contar free tier.

Arquitetura desta prática (serverless)

Item	Uso	Preço unitário	Custo
API Gateway HTTP API	10.000 requisições	US$ 1,00 por milhão	US$ 0,01
Lambda `registrar-chamado`	10.000 invocações, 375 GB-s	US$ 0,20/milhão + US$ 0,0000166667/GB-s	US$ 0,01
EventBridge	10.000 eventos custom	US$ 1,00 por milhão	US$ 0,01
Entrega aos alvos	20.000 entregas, mesma conta	US$ 0,00	US$ 0,00
Lambda `auditar-chamado`	10.000 invocações, 62,5 GB-s	idem acima	US$ 0,00
SNS	1.000 publicações + 1.000 e-mails	US$ 0,50/milhão req; US$ 2,00/100 mil e-mails	US$ 0,02
CloudWatch Logs	bem abaixo de 1 GB	~US$ 0,50/GB ingerido	US$ 0,01
Total			**≈ US$ 0,06/mês**

Com o free tier (1 milhão de invocações e 400 mil GB-s de Lambda por mês, 1 milhão de requisições SNS, 1.000 e-mails) o custo real fica praticamente em zero neste volume.

API em container sempre ligada atrás de load balancer

Item	Uso	Preço unitário	Custo
Fargate, 1 task 0,25 vCPU / 0,5 GB	730 h	US$ 0,0405/vCPU-h e US$ 0,00445/GB-h	US$ 9,01
Application Load Balancer	730 h	US$ 0,0225/h	US$ 16,43
LCUs do ALB	mínimo com esse tráfego	US$ 0,008/LCU-h	poucos dólares
Total			**≈ US$ 25 a US$ 30/mês**

Com duas tasks para redundância, o total sobe para cerca de US$ 34 a US$ 39.

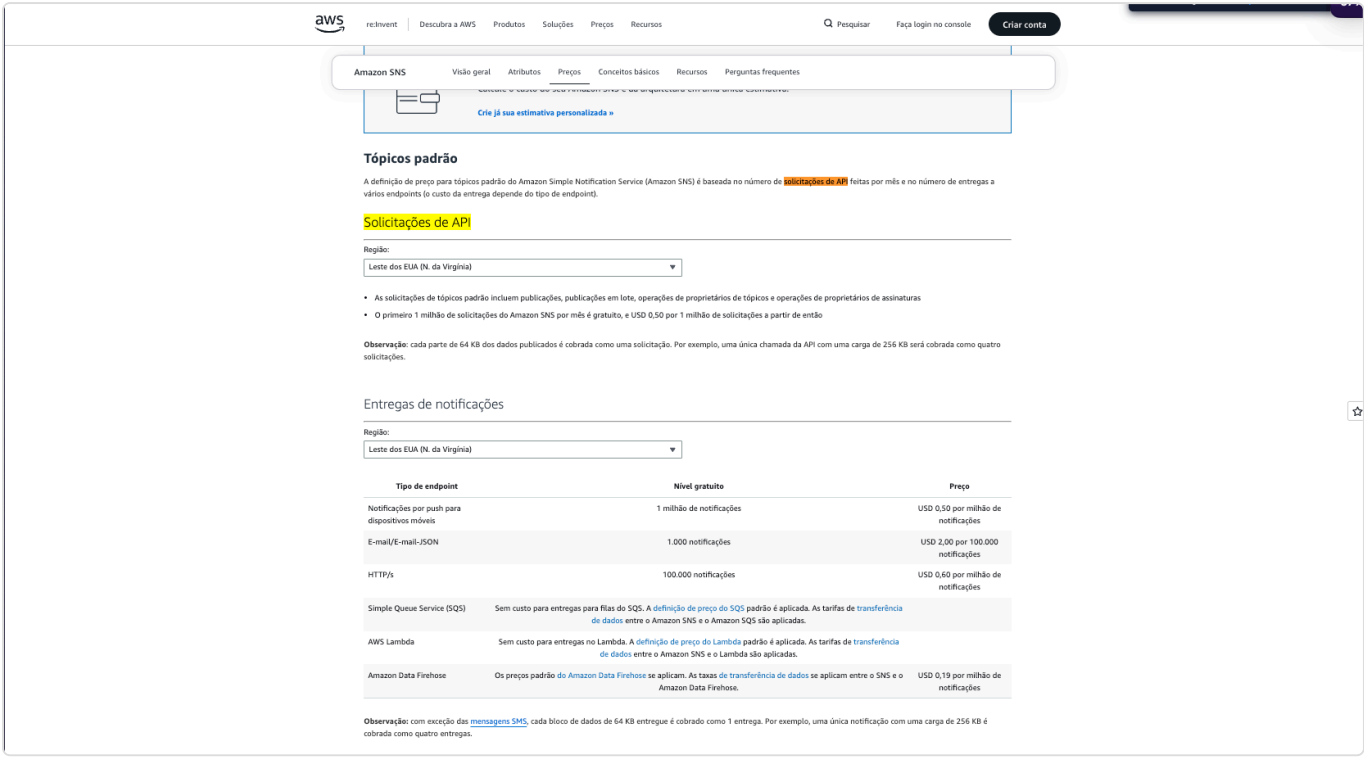
Comparação: cerca de **US$ 0,06 contra US$ 25**, umas 400 vezes mais caro para o mesmo volume. A razão é o modelo de cobrança: o container paga por **tempo ligado**, não por chamado. Dez mil chamados por mês dão aproximadamente 14 por hora, então na maior parte das 730 horas a CPU fica ociosa e sendo paga. O ALB sozinho custa mais de 200 vezes o total da arquitetura serverless.

O container compensaria em volume alto e constante, quando o preço por requisição do serverless ultrapassa o custo fixo. E ele tem duas vantagens que não aparecem na conta: latência previsível, sem cold start (no nosso primeiro teste o cold start custou 356 ms, E08a), e ausência do limite de 15 minutos de execução da Lambda.

Fontes de preço, consultadas em 29/10/2026

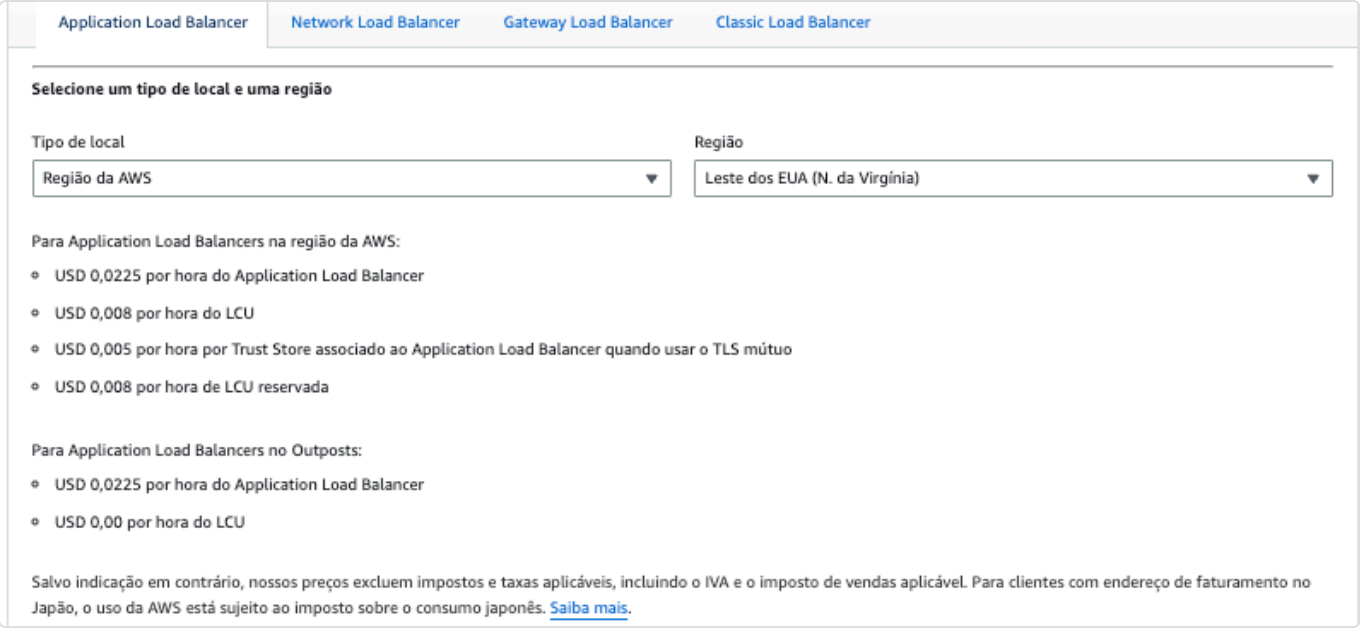
- AWS Lambda pricing — US$ 0,20 por milhão de requisições e US$ 0,0000166667 por GB-s (x86, N. Virginia)
- Amazon API Gateway pricing — US$ 1,00 por milhão de requisições HTTP API
- Amazon EventBridge pricing — US$ 1,00 por milhão de eventos custom; entrega a serviços na mesma conta sem cobrança
- Amazon SNS pricing — tópicos padrão, Leste dos EUA (N. da Virgínia): primeiro 1 milhão de solicitações gratuito e US$ 0,50 por milhão depois; e-mail/e-mail-JSON com 1.000 notificações gratuitas e US$ 2,00 por 100 mil
- AWS Fargate pricing — US$ 0,0405/vCPU-h e US$ 0,00445/GB-h
- Elastic Load Balancing pricing — Application Load Balancer na região Leste dos EUA (N. da Virgínia): US$ 0,0225 por hora do balanceador e US$ 0,008 por hora de LCU

Capturas das páginas de preço



E20 · Página de preços do Amazon SNS, tópicos padrão, região Leste dos EUA (N. da Virgínia)

A captura confirma os dois valores usados na estimativa, com a região selecionada nos dois seletores: **USD 0,50 por 1 milhão de solicitações** de tópico padrão, depois do primeiro milhão gratuito, e **USD 2,00 por 100.000 notificações** de e-mail, com nível gratuito de 1.000 notificações.



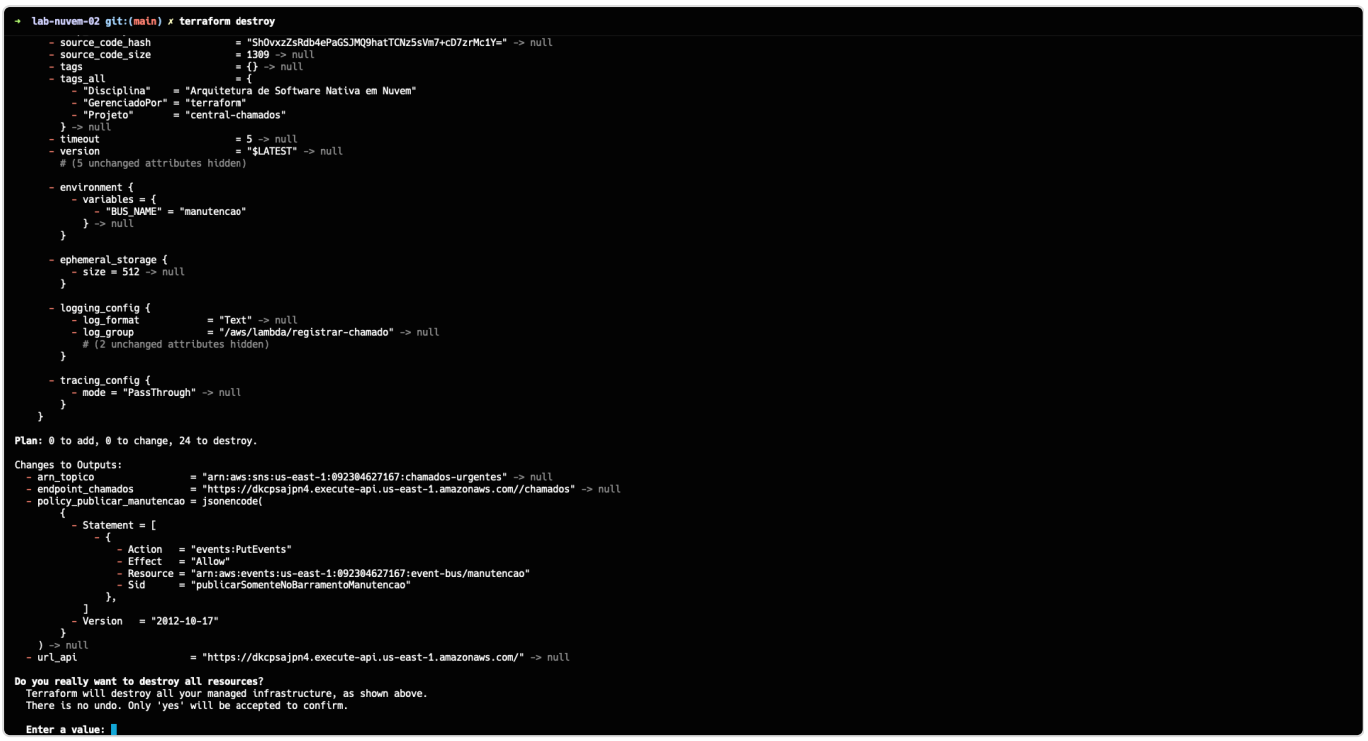
E21 · Página de preços do Elastic Load Balancing, aba Application Load Balancer, região Leste dos EUA (N. da Virgínia)

A captura confirma os dois valores usados na estimativa do cenário em container: **USD 0,0225 por hora** do Application Load Balancer e **USD 0,008 por hora de LCU**, com o tipo de local em Região da AWS e a região em N. da Virgínia.

Com as duas capturas, todos os preços usados no §7 estão confirmados nas páginas oficiais da AWS para us-east-1.

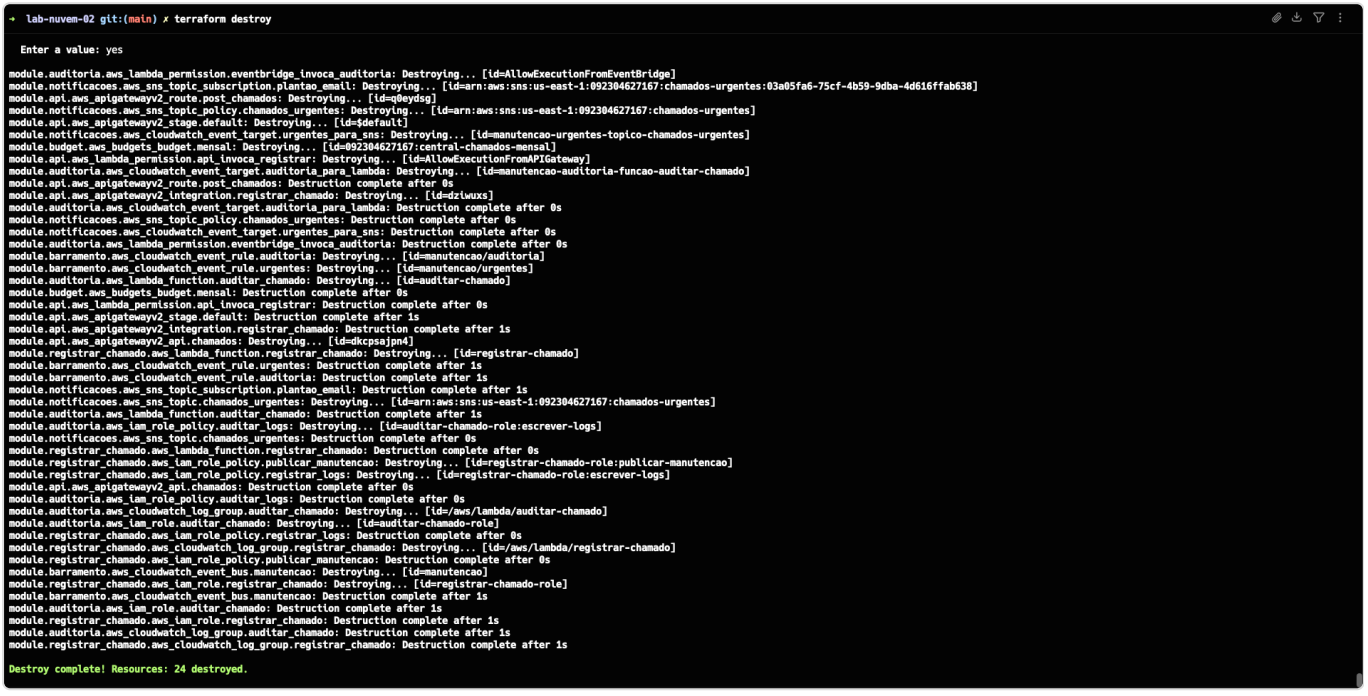
6. Limpeza

Todos os recursos foram removidos com `terraform destroy`, o que evita cobrança depois da entrega.



E18a · Plano do destroy: 24 recursos a destruir, com os outputs que serão removidos

O plano confirma **24 recursos a destruir** e nenhum a criar ou alterar. Aparecem a função `registrar-chamado` com `timeout = 5`, a variável `BUS_NAME = manutencao`, as tags do projeto e a policy `publicarSomenteNoBarramentoManutencao` nos outputs.



E18b · Destroy complete! Resources: 24 destroyed

A saída mostra a remoção em ordem de dependência, módulo por módulo: primeiro as permissões e integrações, depois as regras e alvos, depois funções, tópico, API, roles e log groups, e por fim o

barramento. Termina com `Destroy complete! Resources: 24 destroyed`.

O alerta de orçamento `central-chamados-mensal` também foi removido, pois estava no estado do Terraform (`module.budget.aws_budgets_budget.mensal`).